



Politechnika Warszawska

Warsaw University of Technology

<https://repo.pw.edu.pl>

Tytuł / Title	Rozwój algorytmów poprawiających skuteczność klasyfikacji w zastosowaniu do danych przestrzennych / Development of Algorithms Improving the Quality of Classification for Geographic Data Applications
Autor / Author	Iwanowski Michał Tadeusz
Promotor / Supervisor	Mulawka Jan
Adres publikacji w Repozytorium URL / Publication address in Repository	https://repo.pw.edu.pl/info/master/WUT33916cdd5e014b268bca2c250c6026a7/
Data opublikowania w Repozytorium / Deposited in Repository on	2023-11-14

Politechnika Warszawska
Wydział Elektroniki i Technik Informacyjnych
Instytut Informatyki

Rok akademicki 2014/2015



PRACA DYPLOMOWA MAGISTERSKA

Michał Iwanowski

Rozwój algorytmów poprawiających skuteczność klasyfikacji w zastosowaniu do danych przestrzennych

Opiekun pracy:

prof. dr hab. inż. Jan Mulawka

Ocena

Podpis Przewodniczącego

Komisji Egzaminu Dyplomowego



<i>Kierunek:</i>	Informatyka
<i>Specjalność:</i>	Inżynieria Systemów Informatycznych
<i>Data urodzenia:</i>	8 czerwca 1989 r.
<i>Data rozpoczęcia studiów:</i>	1 października 2009 r.

Życiorys

Nazywam się Michał Iwanowski, urodziłem się 8 czerwca 1989 r. w Łodzi. Ukończyłem z wyróżnieniem I Liceum Ogólnokształcące im. Mikołaja Kopernika w Łodzi, będąc w klasie o profilu matematyczno-fizycznym. W 2009 roku rozpocząłem studia na Wydziale Elektroniki i Technik Informatycznych Politechniki Warszawskiej, na kierunku Informatyka. Do moich zainteresowań należy literatura, muzyka alternatywna oraz gotowanie.

.....
podpis studenta

Egzamin dyplomowy

Złożył egzamin dyplomowy w dn.
Z wynikiem
Ogólny wynik studiów
Dodatkowe wnioski i uwagi Komisji
.....

Streszczenie

Praca ta poświęcona jest zagadnieniom związanym z wykorzystaniem modeli grupowych do klasyfikacji binarnej. Modele grupowe stanowią istotną kategorię metod uczenia się maszyn, a do najważniejszych jej elementów należy algorytm AdaBoost. W niniejszej pracy zaproponowane zostały modyfikacje tegoż algorytmu, polegające na wprowadzeniu lokalnych wag dla klasyfikatorów bazowych. Wpływ wprowadzonych zmian na skuteczność algorytmu został zweryfikowany eksperymentalnie na przykładzie zestawu geograficznych zbiorów danych. Zbadany został również wpływ wykorzystania metod inżynierii cech na przebieg krzywych uczenia badanych modeli. Przeprowadzone eksperymenty wskazują na poprawę dokładności oraz miary F_1 zmodyfikowanego wariantu algorytmu AdaBoost zarówno w funkcji liczby iteracji algorytmu jak i czasu.

Słowa kluczowe: *boosting, AdaBoost, lokalność, dane geograficzne, GIS, inżynieria cech*

Abstract

Title: *Development of Algorithms Improving the Quality of Classification for Geographic Data Applications*

The subject of this thesis is the usage of ensemble methods in binary classification. Ensemble methods constitute a significant category of machine-learning techniques, and one of its most important elements is the AdaBoost algorithm. This thesis contains a proposed modification of AdaBoost algorithm based on local weighting of base classifiers. Introduced changes were experimentally verified using a group of geographic data sets. An additional subject of conducted experiments was the effect of geographic feature engineering methods on the shape of resulting learning curves of inspected models. Results suggest an improvement in classification accuracy and F -score of the

modified version of the algorithm, with respect to both iteration count as well as time.

Keywords: *boosting, AdaBoost, locality, geographic data, GIS, feature engineering*

Spis treści

1	Wstęp	3
1.1	Rozwój algorytmów uczenia się maszyn	3
1.2	Zakres i cel pracy	5
2	Zagadnienia tworzenia modeli klasyfikatorów	7
2.1	Uczenie z nadzorem	7
2.2	Klasyfikacja w uczeniu maszynowym	10
2.2.1	Rozważania wstępne	10
2.2.2	Dekompozycja problemów wieloklasowych	11
2.2.3	Klasyfikacja binarna	12
2.3	Obciążenie i wariancja klasyfikatora	14
3	Metody grupowe i dane geograficzne	17
3.1	Bagging	17
3.2	Boosting	19
3.2.1	AdaBoost	19
3.2.2	Inne algorytmy	21
3.3	Dane geograficzne	22
3.3.1	Źródła danych	22
3.3.2	Zastosowania danych geograficznych	23
3.3.3	Metody przetwarzania danych geograficznych	24
4	Propozycja algorytmu boostingu lokalnego	26
4.1	Motywacja oraz intuicje	26
4.2	Metody realizacji	27
4.2.1	Dodatkowy klasyfikator	27
4.2.2	Wybór regresji logistycznej	28
4.2.3	Wprowadzenie dodatkowej funkcji łączącej	30
5	Implementacja tworzonego oprogramowania	32
5.1	Wykorzystane technologie	32

5.2	Architektura proponowanego rozwiązania	33
6	Weryfikacja eksperymentalna	36
6.1	Zbiory danych	36
6.1.1	<i>London01, London02, London03</i>	36
6.1.2	<i>California01, California02, California03</i>	36
6.2	Cele i metody	37
6.2.1	Inżynieria cech przestrzennych	37
6.2.2	Lokalne warianty boostingu	39
6.3	Scenariusze eksperymentów	39
6.4	Wyniki eksperymentów	40
6.5	Analiza wyników	59
7	Podsumowanie pracy	61
7.1	Wnioski końcowe	61
7.2	Dalszy rozwój aplikacji	61

1 Wstęp

1.1 Rozwój algorytmów uczenia się maszyn

W ostatnich latach intensywnie rozwijane są różne obszary sztucznej inteligencji. Na znaczeniu rośnie uczenie maszynowe, jako obszar wiedzy na pograniczu informatyki, statystyki oraz sztucznej inteligencji. Szybki rozwój tej dziedziny, obecnej w literaturze już od lat 50-tych dwudziestego wieku [1], związany jest w dużym stopniu ze wzrostem ilości przetwarzanych automatycznie danych we wszystkich sferach działalności człowieka. Spadek cen nośników umożliwia gromadzenie i przechowywanie takich wolumenów informacji, których analiza, przetworzenie i wykorzystanie przestało być możliwe do realizacji w sposób inny niż automatyczny. Bardzo często spotykanym terminem opisującym ten proces jest *Big Data* [2]. Termin ten nie ma ścisłej definicji i jest często używany w żargonie marketingowym w celu reklamowania rozwiązań (najczęściej rozproszonych systemów) zdolnych do przechowywania i przetwarzania danych niemożliwych do przetwarzania przy użyciu pojedynczych komputerów oraz tradycyjnych metod. Jednym z kryteriów mówiących o tym, kiedy można mówić o danych jako o *Big Data* jest tak zwane kryterium 3V [3] stawiające następujące postulaty:

- rozmiar (ang. *volume*) - dane są zbyt duże by mieściły się na standardowych dyskach twardych pojedynczych komputerów,
- różnorodność (ang. *variety*) - dane pochodzą z różnych źródeł, są niejednorodne i często słabo ustrukturyzowane,
- prędkość (ang. *velocity*) - tempo napływu nowych danych jest znaczące i utrudnia ich analizowanie.

Na uwagę zasługuje tutaj drugi z powyższych punktów, którego istotność można zaobserwować w dużym zróżnicowaniu typów danych poddawanych analizie - od tabelarycznych danych zawierających wartości liczbowe, poprzez dane tekstowe aż po dane geograficzne, zawierające informacje o lokalizacji

przestrzennej obiektów lub wydarzeń. Tym ostatnim zagadnieniom poświęcono w niniejszej pracy szczególną uwagę.

Zjawiskiem przyczyniającym się do nagłego wzrostu źródeł danych jest *Internet przedmiotów* (ang. *Internet of Things* [4]). Według tej koncepcji, różnego typu urządzenia osadzone w maszynach przemysłowych bądź w urządzeniach codziennego użytku (np. elementach instalacji oświetleniowych i grzewczych, sensorach kontroli dostępu do budynków oraz urządzeniach medycznych) komunikujące się ze sobą poprzez sieć Internet wprowadzają nowy wymiar w automatyzacji i tworzą przestrzeń do tworzenia się złożonych, inteligentnych systemów.

Postępująca automatyzacja i optymalizacja procesów towarzyszących nam w różnych dziedzinach gospodarki i życia codziennego niesie ze sobą wzrost wymagań dotyczących takich systemów, szczególnie w kontekście ich “inteligentnego” działania. Tak więc tradycyjne, niezmiennie w czasie algorytmy ustępują miejsca systemom uczącym się, które zwiększają swoją skuteczność pod wpływem przetwarzania coraz większych zbiorów danych.

Uczenie maszynowe, jako dziedzina której przedmiotem są właśnie takie algorytmy, zawiera w sobie dużą liczbę problemów oraz paradygmatów myślowych, których korzenie tkwią w naukach statystycznych, logice, biologii oraz różnych obszarach matematyki. Istotną częścią uczenia maszynowego jest tak zwane uczenie z nadzorem oraz zagadnienie klasyfikacji. Problem ten występuje wtedy, kiedy system musi podejmować decyzje polegające na rozróżnianiu różnego typu obiektów (przykładowo transakcji uczciwych od nieuczciwych bądź zdjęć kotów od zdjęć psów). Opracowanych zostało wiele algorytmów służących do klasyfikacji, a jednymi z najistotniejszych przez wzgląd na ich jakość oraz wydajność są modele grupowe.

W kategorii modeli grupowych występują różnego typu algorytmy, których działanie opiera się na złożeniu pewnej liczby modeli innego typu oraz odpowiednim połączeniu ich wyników. Algorytm *AdaBoost* [5] jest przykładem modelowania grupowego należącym do szerszej kategorii metod znanych zbiorczo pod nazwą *boosting* [6]. Metody te, a w szczególności algorytm *AdaBoost*, wykorzystują iteracyjny proces rozszerzania finałowego klasyfikatora o kolejne klasyfikatory bazowe. Odpowiednie połączenie pewnej liczby klasyfi-

katorów bazowych pozwala na uzyskanie wysokiej jakości finałowego modelu.

Algorytmy klasyfikacji operują na różnego typu danych, pośród których istotną kategorię stanowią dane geograficzne. Dane te zwyczajowo przechowywane są i przetwarzane w systemach typu *GIS*¹, które to umożliwiają efektywne wyszukiwanie oraz wykonywanie operacji typowych dla danych przestrzennych. Do operacji takich należą między innymi obliczanie odległości pomiędzy obiektami oraz ewaluacja predykatów przestrzennych określających między innymi sąsiedztwo oraz nakładanie się obiektów.

1.2 Zakres i cel pracy

Jak przedstawiono w poprzednim punkcie algorytmy uczenia się maszyn stanowią istotną grupę oprogramowania o dużym znaczeniu praktycznym, cechującą się dużą dynamiką rozwoju. Dlatego warto rozwijać badania w tej dziedzinie, czemu poświęcona została niniejsza praca. Dotyczy ona rozwoju metod uczenia się z nadzorem w aspekcie uwzględniającym modele grupowe.

Celem badań opisanych w niniejszej pracy jest opracowanie metod poprawiających skuteczność klasyfikacji, zwłaszcza w zastosowaniu do danych przestrzennych. Aby cel ten osiągnąć, rozważonych i poddanych weryfikacji eksperymentalnej będzie szereg koncepcji, które można przypisać do jednej z dwóch kategorii:

- inżynieria cech: z wykorzystaniem systemów typu *GIS* zbadano wybrane metody wyprowadzania cech pochodnych, które wpływają korzystnie na dokładność klasyfikacji,
- modyfikacje algorytmu *AdaBoost*: w oparciu o charakterystyczne cechy danych geograficznych wprowadzono modyfikację do najbardziej popularnego algorytmu należącego do kategorii *boosting* i zbadano ich wpływ na dokładność klasyfikacji.

Dla realizacji powyższych celów utworzony będzie zestaw bibliotek zawierających implementacje różnych wariantów algorytmu *AdaBoost* oraz zautomatyzowane środowisko testowe. Utworzony będzie także zestaw skryptów służą-

¹ang. *Geographic Information System*

cych do inżynierii pochodnych cech przestrzennych. Przeprowadzony zostanie zestaw eksperymentów mających na celu zbadanie skuteczności proponowanych metod zarówno pod kątem dokładności predykcji jak i wydajności.

2 Zagadnienia tworzenia modeli klasyfikatorów

2.1 Uczenie z nadzorem

W uczeniu się maszyn łączy się wiele paradygmatów algorytmicznych, które różnią się znacznie pod względem ujęcia problemu. Natomiast ich cechą wspólną jest realizowanie następującego celu: utworzenie systemu, który jest w stanie modyfikować pewne swoje elementy pod wpływem danych wejściowych, tak aby maksymalizować swoją skuteczność względem zadanych kryteriów. Jednym z paradygmatów pozwalających realizować tego typu zadanie jest uczenie z nadzorem (ang. *supervised learning*). W tym ujęciu, zadaniem algorytmu jest wyznaczenie pewnej funkcji (nazywanej także *hipotezą* [7]) na podstawie danych trenujących, wyposażonych w etykiety.

Na dane trenujące składa się zbiór instancji nazywanych *przykładami trenującymi* składających się z par (*obiekt wejściowy, etykieta*). Zbiór trenujący T złożony z n przykładów można formalnie opisać następująco:

$$T = \{(\mathbf{x}_1; y_1), \dots, (\mathbf{x}_n; y_n)\} \quad (2.1)$$

W powyższym równaniu $\mathbf{x}_i$ oznacza i -ty obiekt wejściowy, a y_i i -tą etykietę. W zależności od typu etykiety możemy wyróżnić dwie główne kategorie problemów w uczeniu z nadzorem:

- regresja - gdy etykietą jest liczba rzeczywista,
- klasyfikacja - gdy etykietą jest dyskretna wartość pochodząca ze skończonego zbioru dopuszczalnych wartości.

Etykietę można interpretować jako prawdziwą wartość funkcji, którą algorytm będzie odwzorowywał. W oparciu o dane trenujące oraz wybraną funkcję błędu (określającą "karę" dla algorytmu za niepoprawne predykcje) algorytm tworzy *model* danych trenujących jako funkcję h :

$$h(\mathbf{x}) : X \ni \mathbf{x} \mapsto y \in Y \quad (2.2)$$

Powyższa funkcja przyporządkowuje obiektom wejściowym etykiety, a jej skuteczność mierzona jest przy użyciu wybranej metryki na zbiorze danych o identycznej strukturze jak zbiór trenujący, nazywanym zbiorem testowym. Metrykę traktować można jako funkcję wyższego rzędu, której pierwszym argumentem jest funkcja reprezentująca model, drugim zbiór testowy, a dziedziną najczęściej pewien podzbiór liczb rzeczywistych:

$$M(h, S) : (h, S) \mapsto m \in [0, 1] \quad (2.3)$$

Przykładem popularnej metryki występującej w problemie regresji jest błąd średniokwadratowy (ang. *mean squared error*):

$$MSE(h, S) = \frac{1}{n} \sum_{\mathbf{x}, y \in S} (h(\mathbf{x}) - y)^2 \quad (2.4)$$

Do istotnych metryk w problemie klasyfikacji należą między innymi dokładność, czułość, precyzja, a także miara F (ang. *F-score* lub *F₁-score*), które omówione zostały szczegółowo w dalszych punktach.

Wykorzystanie odrębnego zbioru testowego dla oceny modelu w uczeniu nadzorowanym jest ściśle związane z problemem nadmiernego dopasowania (ang. *overfitting*). Problem ten występuje wtedy, kiedy model generuje dokładne predykcje dla danych trenujących (dobrze się do nich dopasowuje), ale posiada słabą zdolność generalizacji na nowe dane. Przyczyn tego zjawiska może być wiele, a mechanizm jego powstawania poddany został analizie w dalszej części.

W większości praktycznych przykładów uczenia z nadzorem obiekt wejściowy $\mathbf{x}_i$ ma strukturę wektora:

$$\mathbf{x}_i = \langle x_{i,1}, \dots, x_{i,m} \rangle \quad (2.5)$$

Jego składowe $x_{i,k}$ nazywane są cechami bądź atrybutami przykładu. Analogicznie do etykiet, cechy mogą mieć charakter ciągły lub dyskretny a ich liczba jest ściśle związana ze specyfiką danego problemu.

Znanych jest wiele różnych algorytmów uczenia z nadzorem. Chociaż celem niniejszej pracy nie jest ich przegląd ani ocena, to jednak wartych odno-

towania jest kilka cech je różnicujących:

- typ etykiet przykładów (algorytmy klasyfikacji oraz regresji),
- typ i liczba wspieranych cech,
- interpretowalność otrzymywanych modeli (np. modele czarnoskrzynkowe i białoskrzynkowe)²,
- złożoność obliczeniowa i pamięciowa (tworzenia modelu oraz generowania predykcji),
- moc zbioru możliwych do nauczenia hipotez,
- zdolność do generalizacji.

Do najbardziej znanych algorytmów w uczeniu się maszyn zaliczyć można z pewnością sieci neuronowe [8], drzewa decyzyjne [9], maszyny wektorów nośnych (wspierających) [10], modele oparte na regresji liniowej oraz logistycznej, a także dużą liczbę metod grupowych (opisanych dalej szczegółowo).

Istotnym spostrzeżeniem dotyczącym różnorodności metod uczenia się maszyn, sformułowanym w latach 90-tych, jest tak zwane twierdzenie NFL [11] (ang. *No Free Lunch theorem*). Płynącym z niego wnioskiem jest fakt, że żadna z metod optymalizacji nie daje w sposób uniwersalny lepszych rezultatów jeżeli rezultaty te zostaną uśrednione po zbiorze wszystkich możliwych problemów. W praktyce okazuje się, że pewne algorytmy znane są ze swojej ponadprzeciętnej skuteczności, co tłumaczyć można ich dobrym dopasowaniem do konkretnych, często występujących realnych problemów. Skuteczne rozwiązywanie nowych problemów wymaga jednak wielu eksperymentów a dobór właściwego algorytmu oraz jego parametrów pozostaje ciągle procesem w dużej mierze zależnym od intuicji analityka.

²Modele czarnoskrzynkowe pozbawione są cechy interpretowalności przez człowieka i działają na zasadzie "czarnej skrzynki" o ukrytej strukturze wewnętrznej. Modele białoskrzynkowe są pod tym względem ich przeciwieństwem.

2.2 Klasyfikacja w uczeniu maszynowym

2.2.1 Rozważania wstępne

Klasyfikacja jest jednym z najpowszechniejszych problemów uczenia z nadzorem. Etykiety przykładów trenujących należą tutaj do skończonego zbioru $\mathcal{C}$, nazywanego zbiorem dopuszczalnych klas.

$$y_i \in \mathcal{C} = \{c_1, c_2, \dots, c_K\} \quad (2.6)$$

Zadaniem algorytmu klasyfikacyjnego jest utworzenie modelu, tzn. takiej reprezentacji zbioru trenującego, przy pomocy której możliwe jest skuteczne przydzielanie przykładów pozbawionych etykiet do właściwej klasy. Znane są również algorytmy klasyfikacji wieloetykietowej, w której przykłady należeć mogą do kilku klas jednocześnie, jednak zagadnienie to wykracza poza zakres niniejszej pracy. Do przykładowych zastosowań klasyfikacji należą między innymi:

- rozpoznawanie obrazu oraz widzenie maszynowe,
- kategoryzacja dokumentów,
- filtrowanie spamu,
- rozpoznawanie mowy oraz pisma odręcznego,
- przetwarzanie języka naturalnego,
- realizacja systemów ekspertowych [12],
- wykrywanie oszustw i nadużyć,
- diagnostyka medyczna,
- wspomaganie inżynierii farmakologicznej,
- klasyfikacja biologiczna.

Od algorytmów klasyfikacji wymaga się różnych cech w zależności od obszaru zastosowań. W diagnostyce medycznej bardzo często priorytetem jest *dokładność* wyrażona przy pomocy metryk uwzględniających specyfikę zadania - przykładowo omyłkowe zdiagnozowanie poważnego schorzenia u zdrowego pacjenta jest mniej "kosztowne" niż zaklasyfikowanie osoby chorej jako zdrowej. W systemach czasu rzeczywistego kontrolujących jakość wytwarzanych wyrobów istotna jest wydajność klasyfikatora rozumiana jako czas potrzebny do uzyskania decyzji. Z kolei w taksonomii biologicznej oraz naukach ekonomicznych często wymaga się interpretowalności modeli, co dyskwalifikuje pewne algorytmy charakteryzujące się wysoką dokładnością, ale działające na zasadzie czarnej skrzynki³.

Problemy klasyfikacji można podzielić na:

- binarne oraz
- wieloklasowe

w zależności od mocy zbioru $\mathcal{C}$. W klasyfikacji binarnej przyjęło się oznaczać dwie występujące tam klasy jako 1 (klasa pozytywna) oraz -1 (klasa negatywna). Klasę pozytywną najczęściej łączy się z występowaniem danego zjawiska (oszustwo, chory pacjent), a klasę negatywną z jego brakiem.

2.2.2 Dekompozycja problemów wieloklasowych

Klasyfikacja wieloklasowa (będąca problemem odmiennym od klasyfikacji wieloetykietowej, wspomnianej wcześniej) jest zagadnieniem bardziej złożonym, jednak dającym się zredukować do skończonej liczby problemów klasyfikacji binarnej. Redukcję taką można przeprowadzić jedną z dwóch metod.

Pierwsza z metod, zwana *one-vs.-rest* [13] polega na utworzeniu jednego klasyfikatora binarnego dla każdej klasy, traktując przykłady do niej należące jako przykłady pozytywne, a wszystkie pozostałe jako negatywne. Pożądane jest, aby klasyfikator bazowy zwracał liczbę rzeczywistą określającą "prawdopodobieństwo" przynależności przykładu do klasy pozytywnej, a nie samą etykietę klasy. W przeciwnym wypadku dojść może do niejednoznaczności

³Do przykładów należą sieci neuronowe oraz lasy losowe.

i przewidywania kilku klas dla danego przykładu, w problemie nie będącym klasyfikacją wieloetykietową. Wynik klasyfikacji tą metodą określa się jako etykietę klasyfikatora zwracającego najwyższy wynik:

$$\hat{y} = \arg \max_{c \in \mathcal{C}} h_c(\mathbf{x}) \quad (2.7)$$

Z kolei druga z metod, zwana *one-vs.-one* [13] polega na utworzeniu $\frac{K(K-1)}{2}$ klasyfikatorów binarnych, z których każdy otrzymuje przykłady należące do pary klas w oryginalnym problemie i zostaje wytrenowany aby odróżniać jedne od drugich. Podczas generowania predykcji, stosuje się mechanizm głosowania w obrębie tak stworzonego zestawu klasyfikatorów i jako ostateczny wynik zostaje zwrócona klasa, która otrzymała największą liczbę etykiet pozytywnych. Zarówno jedna, jak i druga z metod może w pewnych przypadkach prowadzić do niejednoznaczności, gdy przykłady w pewnych obszarach przestrzeni wejściowej otrzymują równą liczbę głosów.

2.2.3 Klasyfikacja binarna

Ze względu na prostotę problemu oraz opisaną w poprzednich punktach możliwość dekompozycji zagadnień wieloklasowych, klasyfikacja binarna jest istotna w inżynierii i analizach porównawczych algorytmów uczenia maszynowego. Osiągi klasyfikatorów binarnych badać można w oparciu o grupę metryk nie posiadających oczywistego uogólnienia na problemy wieloklasowe. Aby uzasadnić potrzebę wprowadzenia takich metryk, rozważyć można przypadek dysproporcji pomiędzy klasą pozytywną a negatywną w zbiorze uczącym. Dysproporcja taka ma miejsce w wielu rzeczywistych problemach, w których klasa pozytywna jest rzadka, ale bardzo istotna. Do przykładów zaliczyć można wykrywanie oszustw w transakcjach kartami płatniczymi lub przesiewową diagnostykę nowotworową. Jeżeli etykiety pozytywne stanowią 1% wszystkich etykiet, wtedy posługując się metryką dokładności, zdefiniowaną jako odsetek poprawnie klasyfikowanych przykładów, model trywialny (zwracający zawsze etykietę -1) uzyskuje 99-procentową dokładność. Ze zjawiskiem tym można walczyć poprzez nadanie wyższej wagi przykładom pozytywnym, jednak nie każdy algorytm klasyfikujący jest w stanie wagi takie

poprawnie obsłużyć.

Aby uchronić się przed uzyskaniem mylącej oceny jakości klasyfikatora binarnego, należy rozważyć grupę dodatkowych metryk. Przez przykład prawdziwie pozytywny (których liczbę oznacza się tp) rozumie się taki, którego prawdziwą etykietą jest 1 i takąż etykietę zwrócił klasyfikator. Analogicznie zdefiniowane są pojęcia przykładów prawdziwie negatywnych, fałszywie pozytywnych i fałszywie negatywnych (których liczby oznacza się odpowiednio tn , fp , fn). Precyzją (ang. *precision*) określa się następujący stosunek:

$$precision = \frac{tp}{tp + fp} \quad (2.8)$$

Wielkość ta oznacza odsetek przykładów prawdziwie pozytywnych spośród wszystkich tych, które jako pozytywne określił klasyfikator. Wartości tej metryki bliskie zeru oznaczają dużą liczbę “fałszywych alarmów”.

Czułość (ang. *recall* lub *sensitivity*) zdefiniowana jest następująco:

$$recall = \frac{tp}{tp + fn} \quad (2.9)$$

Intepretacją czułości jest zdolność klasyfikatora do wykrywania przykładów pozytywnych. Niska czułość wiąże się z dużą liczbą “przeoczonych” przykładów pozytywnych.

Aby połączyć dwie powyższe metryki i wprowadzić jedną wielkość oceniającą klasyfikator binarny, a nieobarczoną niedogodnościami takimi jak dokładność, wprowadzono metrykę nazywaną miarą F_1 [14] (ang. *F-measure*, *F-score* lub *F₁ score*):

$$F_1 = \frac{precision \cdot recall}{precision + recall} \quad (2.10)$$

Miara ta jest średnią harmoniczną precyzji i czułości, a jej zbiorem wartości jest przedział $[0, 1]$. Jest ona szczególnym przypadkiem miary F_β , która dopuszcza nadanie większej wagi precyzji bądź czułości poprzez manipulowanie parametrem β :

$$F_\beta = (1 + \beta^2) \cdot \frac{precision \cdot recall}{(\beta^2 \cdot precision) + recall} \quad (2.11)$$

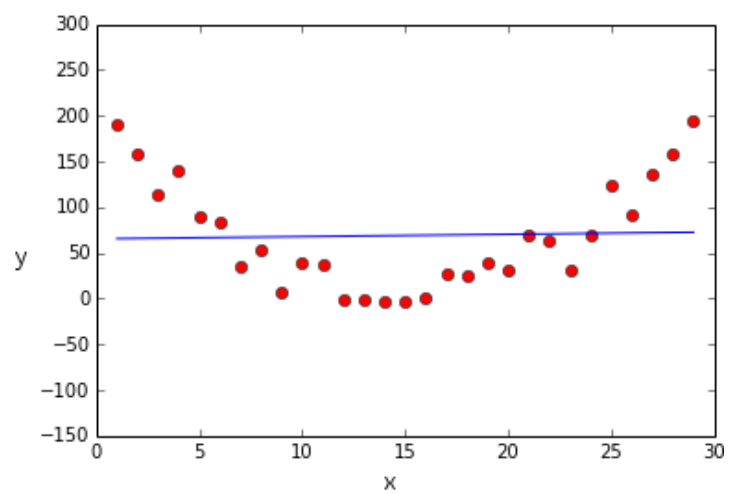
2.3 Obciążenie i wariancja klasyfikatora

Głównym problemem pojawiającym się podczas budowy modeli predykcyjnych (w tym klasyfikatorów) jest zjawisko nadmiernego dopasowania. Występuje ono, gdy tworzony model zbyt mocno odwzorowuje charakterystykę danych treningowych wraz z występującymi w niej szumami. Błąd treningowy (odsetek błędnie klasyfikowanych przykładów w zbiorze treningowym) w takiej sytuacji znacznie spada, co może powodować złudne wrażenie poprawy jakości modelu. Sposobem na zweryfikowanie jakości klasyfikatora jest wyznaczenie błędu testowego, to znaczy odsetku niepoprawnie klasyfikowanych przykładów spoza danych treningowych. Nadmierne dopasowanie jest zjawiskiem dotyczącym szerszej grupy zagadnień uczenia maszynowego niż tylko klasyfikacja. Zagadnieniem, które dobrze nadaje się do graficznej ilustracji nadmiernego dopasowania jest regresja. W problemie tym, etykiety przykładów są liczbami rzeczywistymi, a zadaniem algorytmu jest wyznaczenie funkcji h :

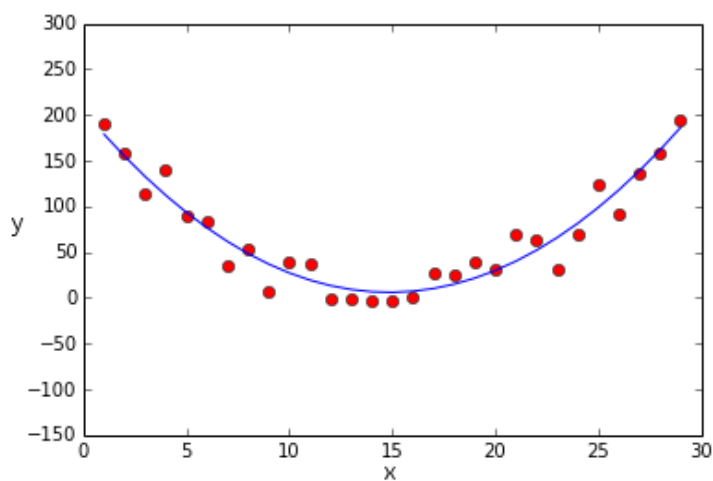
$$h(\mathbf{x}) : \mathbf{x} \mapsto y \in \mathbb{R} \quad (2.12)$$

W użytym przykładzie, przyjęto dla uproszczenia, że przykłady mają tylko jedną cechę, to znaczy wektor $\mathbf{x}$ traktować można jako wielkość skalarną x . Rysunki 2.1-2.3 przedstawiają przypadki niedostatecznego, prawidłowego, oraz nadmiernego dopasowania zależności funkcyjnej (krzywej regresji) do danych treningowych.

Nadmierne oraz niedostateczne dopasowanie są ściśle związane z pojęciami obciążenia oraz wariancji modeli predykcyjnych. O dużym obciążeniu mówimy w sytuacji, gdy model niedostatecznie dokładnie odwzorowuje zależności w danych treningowych. Sytuacja ta ma miejsce na przykład wtedy, gdy postać modelu jest zbyt prosta w stosunku do stopnia skomplikowania zbioru danych. Sytuacją przeciwną jest duża wariancja, która oznacza, że wybór konkretnego zestawu danych treningowych bardzo mocno wpływa na postać modelu. Model taki jest nadmiernie dopasowany, a jego zdolność generalizacji jest niewystarczająca, o czym przekonać się można wyznaczając błąd testowy. Pomiedzy obciążeniem a wariancją występuje kompromis, którego kontrolowanie ułatwiają między innymi techniki określane jako metody

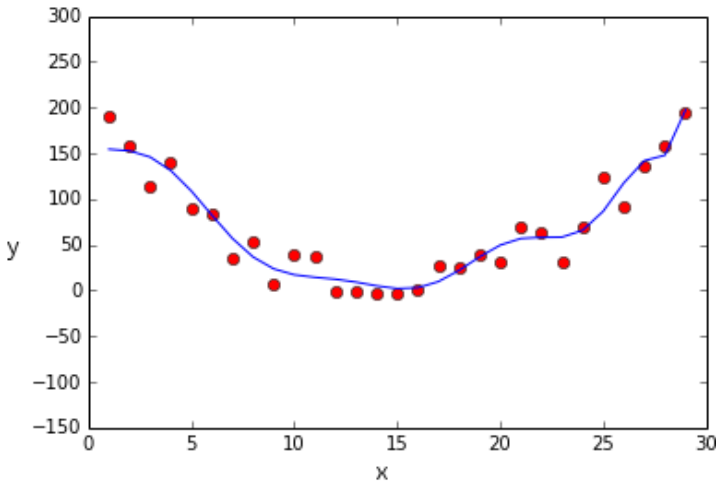


Rysunek 2.1: Przykład niedostatecznego dopasowania



Rysunek 2.2: Przykład prawidłowego dopasowania

grupowe, opisane w kolejnym punkcie.



Rysunek 2.3: Przykład nadmiernego dopasowania

3 Metody grupowe i dane geograficzne

Metody omawiane w tym punkcie oparte są na następującej obserwacji: zbiór modeli, których indywidualna dokładność nie jest satysfakcjonująca, może zostać połączony w celu uzyskania modelu o lepszej charakterystyce. To ogólne stwierdzenie odzwierciedlone jest w następujących zjawiskach:

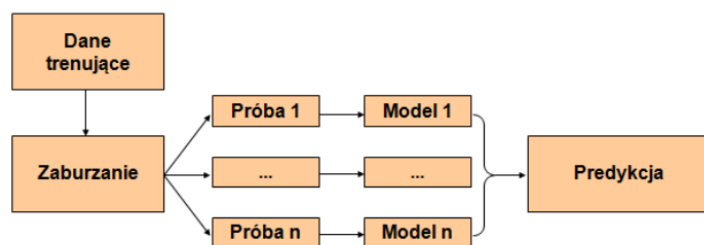
1. Grupa modeli o wysokiej wariancji może zostać połączona w jeden “stabilny” model o lepszej zdolności generalizacji, to znaczy bardziej odporny na nadmierne dopasowanie. Techniki wykorzystujące ten fakt znane są pod nazwą *bagging* [15].
2. Odpowiednia grupa modeli o wysokim obciążeniu może zostać połączona w celu utworzenia modelu bardzo dobrze odwzorowującego zbiór treningowy, przy jednoczesnej kontroli nadmiernego dopasowania. Iteracyjny proces budowy i łączenia takich modeli realizowany jest poprzez zbiór technik określanych jako *boosting* [6].

Skuteczność podejścia grupowego jest szczególnie wysoka dla *niestabilnych* modeli bazowych, to znaczy takich, dla których postać modelu jest bardzo wrażliwa na próbkowanie wzorców trenujących. Przykładem takich modeli są drzewa decyzyjne, które mogą diametralnie zmienić swoją strukturę pod wpływem usunięcia lub dodania do zbioru niewielkiej liczby wzorców. Niestabilność drzew decyzyjnych jest istotna, ponieważ pozwala na osiągnięcie dużej wariancji w grupie klasyfikatorów bazowych - co z kolei prowadzi do zapewnienia wysokiej dokładności całego zespołu klasyfikatorów.

3.1 Bagging

W przebiegu pierwszej z omawianych technik grupowych: *bagging*, której nazwa pochodzi od angielskiego sformułowania *bootstrap aggregating*, generowana jest rodzina zaburzonych zbiorów trenujących $T_1, \dots, T_N$, z których każdy jest wynikiem losowania przykładów ze zwracaniem z pierwotnego zbioru trenującego T . W ten sposób, w każdym z zaburzonych zbiorów trenujących

pewne przykłady mogą występować wielokrotnie, a inne nie pojawić się wcale. Liczność każdego z zaburzonych zbiorów może być taka sama, jak zbioru pierwotnego lub inna, co zależy od rodzaju bazowego algorytmu klasyfikacji oraz rozmiaru dostępnych danych treningowych. Następnie, w oparciu o i -ty zaburzony zbiór budowany jest osobny klasyfikator bazowy h_i . Aby wyznaczyć predykcję otrzymanego w ten sposób klasyfikatora złożonego, należy najpierw dla danego przykładu x wyznaczyć klasy przewidywane przez poszczególne klasyfikatory bazowe, a następnie wybrać najczęściej pojawiającą się klasę. Tak otrzymana etykieta jest więc wynikiem głosowania klasyfikatorów bazowych. Schemat techniki *bagging* przedstawiony został na rys. 3.1.



Rysunek 3.1: Schemat techniki *bagging*

Wyniki eksperymentalne przedstawione w [15] wykazują, że otrzymane w ten sposób predykcje są dla większości zbiorów danych dokładniejsze niż predykcje pojedynczych klasyfikatorów. Dodatkowo, w przypadku danych o dużych rozmiarach wybór grupy niewielkich prób losowych może znacząco ograniczyć czas budowy klasyfikatora względem wykorzystania całości dostępnych danych w charakterze zbioru trenującego. Do ograniczeń techniki *bagging* zaliczyć można fakt, że każdy z klasyfikatorów bazowych jest równouprawniony - nawet pomimo ewentualnych znaczących różnic w ich indywidualnej dokładności. Dodatkowo każdy z przykładów trenujących także traktowany jest jednakowo, pomimo że niektóre z nich sprawiać mogą większą trudność w prawidłowym zaklasyfikowaniu. Zaletą tej metody jest bardzo łatwe zrównoleglenie, która to cecha wynika z faktu, że modele bazowe mogą zostać budowane w sposób całkowicie niezależny.

3.2 Boosting

Pokrewną metodą grupową jest *boosting* [6]. Istnieje wiele konkretnych realizacji tej techniki [5, 16–18], jednak można wyróżnić pewien ogólny zestaw kroków wspólny dla większości meta-algorytmów określanych zbiorczo jako *boosting*:

1. Każdemu z przykładów trenujących $\mathbf{x} \in T$ przypisywana jest identyczna waga początkowa.
2. Budowany jest nowy klasyfikator h_i w oparciu o zbiór trenujący, z uwzględnieniem wag przykładów.
3. Nowy klasyfikator h_i dołączany jest do zbiorczego klasyfikatora H , z wagą odpowiadającą jego dokładności.
4. Po dodaniu klasyfikatora h_i , następuje taka aktualizacja wag przykładów, że przykłady klasyfikowane poprawnie otrzymują wagę zmniejszoną, natomiast przykłady klasyfikowane niepoprawnie - zwiększoną.
5. Jeżeli nastąpił warunek stopu (zwykle jest to maksymalna liczba iteracji lub niedostatecznie wysoki przyrost dokładności klasyfikatora w ostatniej iteracji) działanie algorytmu kończy się i w efekcie otrzymany zostaje złożony klasyfikator H . W przeciwnym wypadku, powrót do punktu 2.

3.2.1 AdaBoost

Wiele spośród znanych algorytmów techniki *boosting* może być traktowanych jako należące do klasy *AnyBoost* [19], która stanowi uogólnienie ich założeń i przedstawia je jako przykłady gradientowego przeszukiwania przestrzeni rozwiązań z użyciem wypukłej funkcji kosztu. Prawdopodobnie najbardziej szeroko rozpowszechnionym z algorytmów realizujących *boosting* jest algorytm *AdaBoost* [5], którego nazwa pochodzi od angielskiego określenia *adaptive boosting*. Algorytm ten został zaproponowany w 1997 roku przez Yoava Freund'a oraz Roberta Schapire [5], którzy później otrzymali za swoją pracę Nagrodę Gödla. Przebieg algorytmu jest następujący:

1. Dany jest zbiór trenujący T zawierający n przykładów, liczba iteracji K oraz początkowy rozkład wag dla przykładów trenujących $D_1(i) = \frac{1}{n}$ dla każdego $i \in \{1, \dots, n\}$.
2. W każdej iteracji $k \in \{1, \dots, K\}$ budowany jest nowy klasyfikator bazowy h_k pochodzący z rodziny klasyfikatorów $\mathcal{H}$, który maksymalizuje wartość bezwzględną różnicy pomiędzy własnym poziomem błędu ϵ_k a wartością $\frac{1}{2}$ (z uwzględnieniem aktualnego rozkładu wag D_k):

$$h_k = \underset{h \in \mathcal{H}}{\operatorname{argmax}} \left| \frac{1}{2} - \epsilon_k \right| \quad (3.1)$$

gdzie

$$\epsilon_k = \sum_{i=1}^n D_k(i) \delta_i \quad (3.2)$$

oraz $\delta_i = 0$ gdy i -ty przykład został błędnie sklasyfikowany ($y_i \neq h_k(\mathbf{x}_i)$), a 1 w przeciwnym wypadku.

3. Jeżeli $\left| \frac{1}{2} - \epsilon_k \right| \leq \beta$, gdzie β jest wybranym wcześniej progiem minimalnej dokładności uzyskiwanych klasyfikatorów, następuje skok do punktu 6.
4. Obliczona zostaje nowa waga α_k :

$$\alpha_k = \frac{1}{2} \ln \frac{1 - \epsilon_k}{\epsilon_k} \quad (3.3)$$

5. Dla każdego **poprawnie sklasyfikowanego** przykładu trenującego $\mathbf{x}_i$ następuje uaktualnienie wagi $D_k(i)$:

$$D_{k+1}(i) = \frac{D_k(i) \exp(-\alpha_k)}{\xi} \quad (3.4)$$

Natomiast dla każdego przykładu **niepoprawnie sklasyfikowanego**:

$$D_{k+1}(i) = \frac{D_k(i) \exp(\alpha_k)}{\xi} \quad (3.5)$$

gdzie ξ jest współczynnikiem normalizującym dobranym dla całej dys-

trybucji w taki sposób, by suma wag wszystkich przykładów wynosiła 1.

6. Zostaje utworzony klasyfikator złożony H :

$$H(\mathbf{x}) = \text{sign}\left(\sum_{k=1}^K \alpha_k h_k(\mathbf{x})\right) \quad (3.6)$$

Dzięki temu, w każdej iteracji algorytmu dołączony zostaje klasyfikator bazowy, który sprawdził się najlepiej w odniesieniu do dystrybucji wag eksponującej przykłady błędnie rozpoznane przez klasyfikatory poprzednie. Algorytm *AdaBoost* może być rozumiany jako minimalizacja wykładniczej funkcji kosztu:

$$\sum_{i=1}^n e^{-y_i f(\mathbf{x}_i)} \quad (3.7)$$

względem poszukiwanej funkcji:

$$f(\mathbf{x}) = \sum_{k=1}^K \alpha_k h_k(\mathbf{x}) \quad (3.8)$$

Ze względu na wykładniczy charakter funkcji kosztu, algorytm *AdaBoost* wykazuje tendencje do nadmiernego dopasowywania się do przykładów wielokrotnie błędnie klasyfikowanych. Cecha ta stanowi problem w odniesieniu do danych zaszumionych, zawierających odsetek przykładów opatrzonych nieprawidłowymi etykietami klas. Może zdarzyć się, że przykłady takie zostaną obciążone wysokimi wagami i w efekcie zdolność generalizacji otrzymanego klasyfikatora złożonego będzie niesatysfakcjonująca. Jedną z możliwości pozwalających częściowo uniknąć nadmiernego dopasowania jest wykorzystanie walidacji krzyżowej w celu ograniczenia liczby iteracji.

3.2.2 Inne algorytmy

Alternatywną realizacją techniki *boosting*, która częściowo rozwiązuje powyższy problem, jest algorytm *LogitBoost* [17]. W algorytmie tym wykorzystana

została odmienna funkcja kosztu:

$$\sum_{i=1}^n \log(1 + e^{-y_i f(\mathbf{x}_i)}) \quad (3.9)$$

Efektom wykorzystania odmiennej funkcji kosztu jest mniejsza wrażliwość algorytmu *LogitBoost* na dane trenujące zawierające szum. Do znanych algorytmów realizujących technikę *boosting* zaliczyć można również algorytm *BrownBoost* [16], który wykorzystuje zestaw równań różniczkowych opisujących ruchy Browna do modelowania wpływu poszczególnych iteracji na zachowanie wynikowego klasyfikatora. Autorzy proponują dwie metody przybliżonego rozwiązywania wyżej wymienionych równań w celu efektywnej realizacji algorytmu. Kolejnym wartym wymienienia wariantem schematu *boosting* jest algorytm *LPBoost* [18] wykorzystujący metodę programowania liniowego w celu optymalnego złączenia klasyfikatorów bazowych. Przez optymalne rozumie się w tym kontekście takie złączenie, które zapewnia maksymalizację marginesu klasyfikacji dla przykładów trenujących. Te oraz liczne niewymienione tu warianty techniki *boosting* nie leżą jednak w zakresie niniejszej pracy.

3.3 Dane geograficzne

W kontekście rozpowszechnienia urządzeń wyposażonych w zdolności geolokalizacyjne (w szczególności odbiorników GPS) oraz szybkiego przyrostu pojemności nośników danych obserwuje się wzrost znaczenia danych geograficznych. Ich analiza niesie ze sobą możliwości i wyzwania nieobecne w przypadku innych typów danych. Są one istotne również w kontekście uczenia maszynowego oraz problemu klasyfikacji.

3.3.1 Źródła danych

Do najistotniejszych źródeł cyfrowych danych geograficznych zaliczyć można:

- otwarte, zdigitalizowane mapy możliwe do pobrania bądź dostępne jako

usługa (np. *GoogleMaps*⁴ lub *OpenStreetMap*⁵),

- zbiory danych o placówkach i wydarzeniach zawierające tzw. *geotag*, czyli koordynaty geograficzne złożone z długości oraz szerokości geograficznej (oraz opcjonalnie wysokości n.p.m. i stempla czasowego),
- sygnały nadawane w czasie rzeczywistym przez ludzi lub pojazdy znajdujące się w ruchu,
- zdjęcia satelitarne.

Na szczególną uwagę wśród zjawisk przyczyniających się do wzrostu znaczenia danych geograficznych zasługuje rozpowszechnienie odbiorników GPS zintegrowanych w telefonach komórkowych oraz aplikacji mobilnych wykorzystujących te dane w celach informacyjnych lub marketingowych.

3.3.2 Zastosowania danych geograficznych

Dane geograficzne oraz odpowiednie techniki ich przetwarzania umożliwiają działanie wielu bardzo istotnych rozwiązań technologicznych. Wśród najważniejszych wskazać można:

- kontrola ruchu morskiego oraz lotniczego,
- nawigacja samochodowa,
- planowanie zagospodarowania terenu,
- projektowanie sieci energetycznych,
- analiza i wizualizacja stanu gleb oraz roślinności,
- rozmieszczanie nadajników GSM,
- transport i logistyka,
- profilowanie klientów w celach marketingowych.

⁴<https://maps.google.com/>

⁵<https://www.openstreetmap.org/>

Inteligentne przetwarzanie danych geograficznych jest jednym z głównych wyzwań dla algorytmów uczenia maszynowego. Wieloagentowe systemy uczące się, które wykorzystują informacje o wzajemnym położeniu urządzeń stanowią ważny krok w rozwoju *Internetu przedmiotów* [4].

3.3.3 Metody przetwarzania danych geograficznych

Systemy służące do przechowywania i przetwarzania danych geograficznych określane są mianem systemów *GIS* (ang. *Geographic Information System*). Dane przechowywane są najczęściej w plikach o formacie wektorowym lub rastrowym (istnieją także formaty hybrydowe) lub w relacyjnych bazach danych wyposażonych w odpowiednie nakładki. Przykładem takiego rozszerzenia jest system *PostGIS*⁶, który stanowi dodatek dla popularnej relacyjnej bazy danych *PostgreSQL*⁷. Reprezentacja danych przestrzennych w systemach *GIS* realizowana może być przy pomocy:

- punktów, odcinków, łamanych oraz wielokątów,
- bitmap opisujących wartość danej funkcji w przestrzeni (np. zasolenia gleby),
- grafów i sieci.

Systemy *GIS* umożliwiają import i konwersję danych geograficznych pochodzących z różnych źródeł oraz narzędzi takich, jak:

- ewaluacja predykatów przestrzennych (np. opisujących relację przecięcia się lub zawierania obiektów),
- obliczanie odległości oraz pól powierzchni,
- wyszukiwanie pobliskich obiektów,
- znajdowanie najkrótszych ścieżek oraz rozwiązywanie innych problemów grafowych,

⁶<http://postgis.net/>

⁷<http://www.postgresql.org/>

- obliczanie otoczki wypukłej zbioru punktów.

W celu umożliwienia efektywnej realizacji wymienionych wyżej operacji, systemy *GIS* wykorzystują wyspecjalizowane metody wewnętrznej reprezentacji danych przestrzennych, takie jak rekursywne partycjonowanie przestrzeni, R -drzewa, $R+$ drzewa, R^* drzewa, indeksy przestrzenne oraz odpowiednie funkcje mieszające. W kontekście problemu klasyfikacji, dane przestrzenne występują najczęściej jako cechy klasyfikowanych obiektów. Przykładowo, obiekty opatrzone długością i szerokością geograficzną przynależeć mogą do kilku klas, a zadaniem klasyfikatora jest stworzenie funkcji odwzorowującej przestrzeń w zbiór możliwych etykiet. Współrzędne geograficzne mogą w takim wypadku być traktowane jako zwykłe liczby rzeczywiste, bądź też wykorzystane do inżynierii dodatkowych cech (będących przykładowo wynikiem ewaluacji predykatów przestrzennych). Problem ten rozwinięty został w dalszej części pracy.

4 Propozycja algorytmu boostingu lokalnego

4.1 Motywacja oraz intuicje

Jak to zostało pokazane poprzednio, działanie algorytmu *AdaBoost* polega na iteracyjnym dodawaniu do modelu kolejnych klasyfikatorów bazowych wraz z wagami, będącymi funkcją całkowitego błędu danego klasyfikatora bazowego. Wagi przekładają się później na udział poszczególnych klasyfikatorów w predykcji generowanej przez cały model.

Spostrzeżenie, które jest punktem wyjścia do modyfikacji powyższej koncepcji jest następujące: pewne zbiory danych charakteryzują się *niejednorodnością*, rozumianą jako zmienność struktury klas w różnych obszarach dziedziny. Przykładem takiej niejednorodności jest występowanie skupień, w których separacja klas wymaga odmiennych warunków decyzyjnych. Typem danych, który zdaje się być szczególnie podatny na to zjawisko są dane geograficzne, dla których kryteria decydujące o przynależności przykładu do danej klasy zależą mogą silnie od szerokości i długości geograficznej oraz położenia w pobliżu pewnych lokalizacji.

Dane niejednorodne prowadzić mogą do sytuacji, w których *globalna* (uśredniona po całej dziedzinie) ocena jakości klasyfikatorów bazowych jest niewystarczająca. Przykładowo, pewne klasyfikatory bazowe mogą generować bardzo niski błąd w jednych obszarach dziedziny, a bardzo wysoki w innych. Waga klasyfikatora, będąca funkcją błędu całkowitego, ma w takim wypadku trudny do zinterpretowania wpływ na końcową predykcję. Metody lokalne w uczeniu maszynowym oraz klasyfikacji były już badane i okazały się skuteczne w zastosowaniu do pewnych problemów [20, 21]. Proponowaną modyfikacją algorytmu *AdaBoost* jest wprowadzenie do równania 3.6 dodatkowego czynnika $\Gamma(\mathbf{x})$, modyfikującego ostateczną wagę klasyfikatora bazowego w zależności od położenia w przestrzeni:

$$H(\mathbf{x}) = \text{sign}\left(\sum_{k=1}^K \alpha_k \Gamma_k(\mathbf{x}) h_k(\mathbf{x})\right) \quad (4.1)$$

Funkcja $\Gamma_k(\mathbf{x})$ zawiera informację o rozkładzie błędów k -tego klasyfi-

katora bazowego i przyjmuje większe wartości w tych obszarach dziedziny, w których błąd danego klasyfikatora jest mniejszy. Sposób realizacji funkcji Γ przedstawiony został w kolejnych punktach.

4.2 Metody realizacji

4.2.1 Dodatkowy klasyfikator

W przebiegu algorytmu *AdaBoost*, kolejne modele dodawane są do zespołu iteracyjnie, w następujących krokach (dla i -tej iteracji). Pogrubione zostały elementy wprowadzone w celu realizacji boostingu lokalnego.

1. Utwórz nowy model bazowy h_i z uwzględnieniem dystrybucji wag przykładów D_i .
2. Oblicz wagę modelu α_i w funkcji jego całkowitego błędu ϵ_i .
3. **Utwórz pomocniczy klasyfikator e_i , dopasowany do rozkładu błędów klasyfikatora h_i .**
4. Dołącz utworzony model wraz z wagą **oraz modelem pomocniczym** do zespołu.
5. Dokonaj aktualizacji rozkładu wag przykładów.

Model e_i spełniać musi następujące wymagania:

1. zwraca liczbę rzeczywistą z przedziału $[0, 1]$, odpowiadającą “prawdopodobieństwu” wygenerowania poprawnej klasyfikacji przez model bazowy h_i ,
2. posiada niewielki narzut obliczeniowy potrzebny na jego zbudowanie,
3. posiada zwartą reprezentację wewnętrzną (małe wymagania pamięciowe).

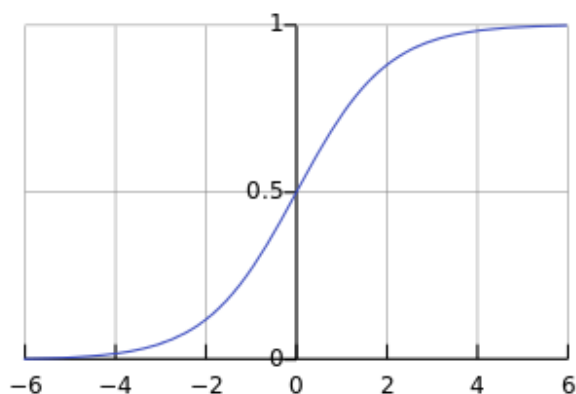
Warunek 1 podyktowany jest chęcią uzyskania ciągłej estymacji dokładności modeli bazowych w funkcji położenia w przestrzeni. Warunki 2-3 wynikają z potrzeby budowania i przechowywania dużej liczby modeli.

4.2.2 Wybór regresji logistycznej

Algorytmem spełniającym powyższe wymagania i w konsekwencji wybranym do realizacji klasyfikatorów pomocniczych jest regresja logistyczna. Algorytm ten opiera się na tzw. funkcji logistycznej, zdefiniowanej następująco:

$$\sigma_t = \frac{1}{1 + e^{-t}} \quad (4.2)$$

Funkcja ta ma własność ciągłego mapowania prostej rzeczywistej w od-cinek $[0, 1]$, a jej wykres przedstawiony został na rys. 4.1.



Rysunek 4.1: Wykres funkcji logistycznej

Jeżeli zmienną t występującą w równaniu 4.2 potraktować jak funkcję liniową atrybutów przykładu trenującego, wtedy można ją wyrazić następująco (rozszerzając wektor $\mathbf{x}$ o dodatkową składową x_0 równą tożsamościowo 1):

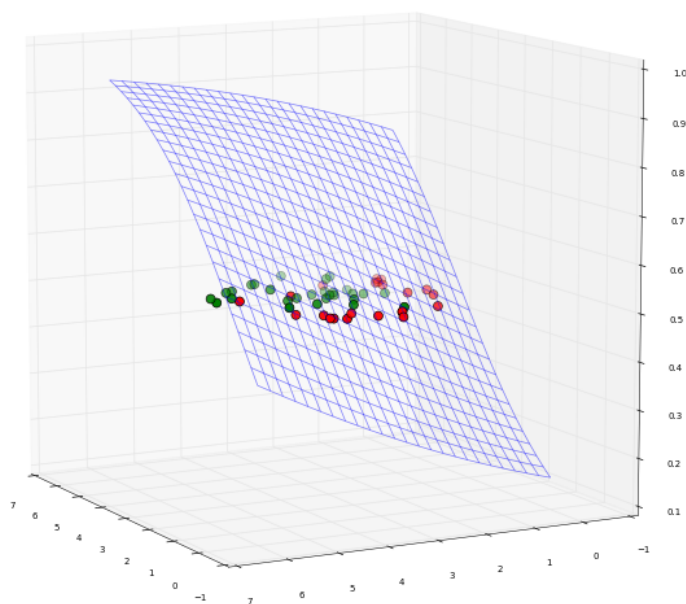
$$t = \beta_0 + \beta_1 x_1 + \cdots + \beta_m x_m = \beta^T \mathbf{x} \quad (4.3)$$

Model regresji logistycznej można wtedy traktować jako funkcję:

$$e(\mathbf{x}) : \mathbf{x} \mapsto \frac{1}{1 + e^{-\beta^T \mathbf{x}}} \in [0, 1] \quad (4.4)$$

Dopasowywanie współczynników modelu w regresji logistycznej jest relatywnie szybkie (w porównaniu do innych modeli klasyfikacyjnych), a jego re-

prezentacja sprowadza się do wektora współczynników β . Z tych powodów regresja logistyczna spełnia sformułowane wcześniej wymagania dotyczące modelu pomocniczego w lokalnym wariancie *AdaBoost*. Rys. 4.2 przedstawia wykres funkcji logistycznej (nazywany też w tym wypadku powierzchnią logistyczną) dla dwuwymiarowej przestrzeni dziedzin.



Rysunek 4.2: Powierzchnia logistyczna

4.2.3 Wprowadzenie dodatkowej funkcji łączącej

Wartość Γ_i występująca w równaniu 4.1 jest funkcją predykcji generowanej przez i -ty model pomocniczy e_i . Bezpośrednie wykorzystanie predykcji klasyfikatora pomocniczego w powyższym równaniu prowadzi jednak do zbyt silnej modyfikacji wag klasyfikatorów bazowych, szczególnie dla wartości $e_i \approx 0$. Stąd potrzeba wprowadzenia funkcji łączącej, której zadaniem jest przetransformowanie surowej predykcji klasyfikatorów pomocniczych w czynnik modyfikujący wagi klasyfikatorów bazowych. Przetestowane zostały następujące funkcje łączące:

- funkcja identycznościowa:

$$\Gamma_i(e_i) = e_i \quad (4.5)$$

- funkcja liniowa:

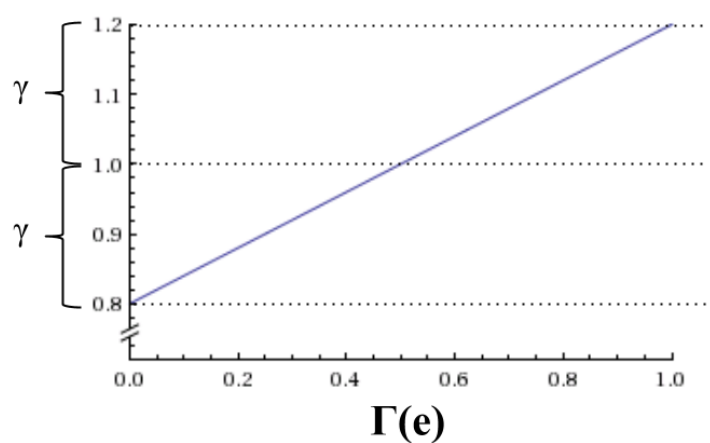
$$\Gamma_i(e_i; \gamma) = \gamma(2e_i - 1) + 1 \quad (4.6)$$

- funkcja nieliniowa (wykorzystująca tangens hiperboliczny):

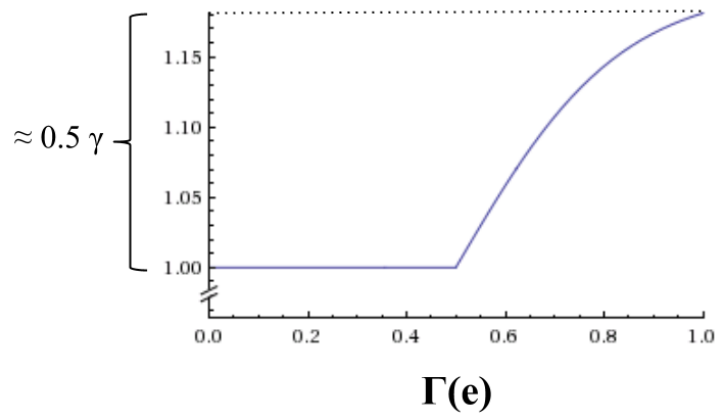
$$\Gamma_i(e_i; \gamma) = \max(\gamma \tanh(e_i - 0.5) + 1, 1) \quad (4.7)$$

Funkcja identycznościowa odpowiada przypadkowi opisanemu w poprzednim paragrafie, mianowicie wykorzystaniu predykcji klasyfikatora pomocniczego wprost. Funkcje liniowa oraz nieliniowa (przedstawione na wykresach 4.3 oraz 4.4) parametryzowane są dodatkowo parametrem γ , regulującym wpływ badanej modyfikacji na działanie algorytmu. Przyjęcie $\gamma = 0$ sprowadza się do przypadku bez modyfikacji. Funkcja liniowa przyjmuje wartości ze zbioru $[1 - \gamma, 1 + \gamma]$, osiągając wartości brzegowe dla predykcji modelu pomocniczego równych odpowiednio 0 oraz 1. Funkcja nieliniowa jest równa 1 dla argumentów z zakresu $[0, 0.5]$ (w efekcie w tym zakresie Γ_i nie wpływa na wagi modeli bazowych), a następnie rośnie w sposób wklęsły osiągając wartość $\tanh(\gamma) \approx 0.5 \cdot \gamma$ dla argumentu równego 1. Motywacją do przetestowania funkcji tej postaci była chęć pozostawienia bez zmian wag klasyfikatorów, dla których model bazowy przewidywał pomyłkę, a stopniowe zwiększenie wag

dla prawdopodobieństwa poprawnej predykcji > 0.5 . Wpływ wyboru każdej z funkcji łączących oraz wartości parametru γ został zbadany eksperymentalnie, a wyniki opisano w dalszej części.



Rysunek 4.3: Wykres liniowej funkcji łączącej



Rysunek 4.4: Wykres nieliniowej funkcji łączącej

5 Implementacja tworzonego oprogramowania

Implementacja bibliotek potrzebnych do eksperymentalnego zweryfikowania skuteczności proponowanych rozwiązań wykonana została w języku *Python 2.7*. Utworzona została zarówno implementacja bazowej wersji algorytmu *Ada-Boost*, jak i jej modyfikacje polegające na boostingu lokalnym. Oprócz tego, stworzone zostało automatyczne środowisko eksperymentalne służące do definiowania i wykonywania scenariuszy testowych, generowania wykresów oraz podsumowań liczbowych. Dodatkowym elementem implementacji jest zestaw skryptów w języku *SQL* wykorzystujących rozszerzenia *GIS* w celu wyprowadzenia pochodnych atrybutów geograficznych (opisane szczegółowo w dalszej części pracy).

5.1 Wykorzystane technologie

Tworząc implementacje poszczególnych elementów składowych proponowanego rozwiązania wykorzystano technologie, które podzielić można na dwie grupy:

- elementy ekosystemu języka *Python* służące do obliczeń numerycznych, uczenia maszynowego oraz wizualizacji danych,
- relacyjna baza danych wraz z rozszerzeniami do przetwarzania danych geograficznych.

Spośród technologii należących do pierwszej z grup wykorzystano następujące narzędzia:

- *NumPy*⁸ - podstawowa biblioteka służąca do obliczeń numerycznych oraz algebry liniowej,
- *SciPy*⁹ - biblioteka zawierająca implementacje narzędzi naukowych i matematycznych,

⁸<http://www.numpy.org/>

⁹<http://www.scipy.org/>

- *Scikit-learn*¹⁰ - biblioteka zawierająca zbiór algorytmów i narzędzi wykorzystywanych w uczeniu maszynowym,
- *matplotlib*¹¹ - biblioteka do tworzenia wykresów oraz wizualizacji,
- *pandas*¹² - biblioteka do manipulacji danymi, zawierająca klasę *Dataframe*, którego funkcjonalność przypomina analogiczną klasę znaną z języka *R*,
- *ipython notebook*¹³ - alternatywny interpreter języka *Python* wyposażony w interaktywne rozszerzenia ułatwiające analizę danych, tworzenie wizualizacji oraz kolejki zadań.

W odniesieniu do relacyjnych baz danych oraz przetwarzania danych geograficznych wykorzystano również:

- *PostgreSQL*¹⁴ - jedna z najpopularniejszych relacyjnych baz danych, utworzona z naciskiem na rozszerzalność oraz zgodność ze standardami,
- *PostGIS*¹⁵ - rozszerzenie typu *GIS* dla bazy danych *PostgreSQL*, pozwalające na przechowywanie oraz przetwarzanie danych przestrzennych.

Zwróćmy uwagę, że wszystkie wymienione wyżej narzędzia należą do kategorii *open-source*.

5.2 Architektura proponowanego rozwiązania

Tworzona implementacja została zaprojektowana z zachowaniem modularności przy użyciu mechanizmu modułów języka *Python*. Struktura modułów wygląda następująco:

- boosting

¹⁰<http://scikit-learn.org/stable/>

¹¹<http://matplotlib.org/>

¹²<http://pandas.pydata.org/>

¹³<http://ipython.org/notebook.html>

¹⁴<http://www.postgresql.org/>

¹⁵<http://postgis.net/>

- localboost
 - * adaboost
 - * localboost
 - * utils
- scripts
- sql

Najistotniejsze składniki implementacji zawierają następujące pliki źródłowe:

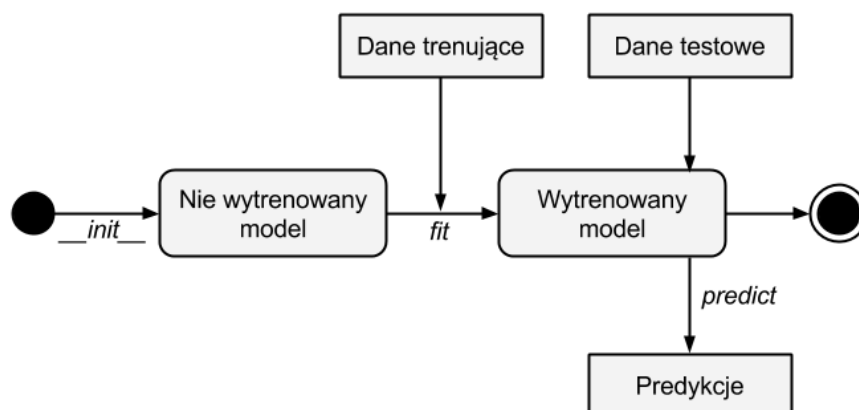
- *adaboost.py* - klasa *AdaBoostEnsemble* będąca realizacją referencyjnego (niezmodyfikowanego) algorytmu,
- *localboost.py* - klasa *LocalBoostEnsemble* będąca realizacją algorytmu po modyfikacjach lokalnych,
- *utils.py* - moduł zawierający zbiór funkcjonalności wykorzystywanych w testowaniu, obliczaniu metryk oraz wizualizacji wyników.

Klasy *AdaBoostEnsemble* oraz *LocalBoostEnsemble* zaprojektowane zostały w oparciu o interfejs oraz sposób użycia stanowiący standard biblioteki *scikit-learn*. Klasy te posiadają następujące metody:

- *__init__(...)* - konstruktor przyjmujący zestaw metaparametrów modelu, tworzący nie wytrenowany model,
- *fit(X, y)* - metoda realizująca proces treningowy w oparciu o macierz predyktorów *X* oraz wektor etykiet *y*,
- *predict(X)* - metoda generująca predykcje dla nowych danych o macierzy predyktorów *X*,
- *staged_predict(X)* - metoda zwracająca generator¹⁶, pozwalający na uzyskanie częściowych predykcji ze wszystkich etapów uczenia (tzn. po każdym dołączonym modelu bazowym), a w konsekwencji umożliwiającą wykreślenie krzywej uczenia.

¹⁶Wbudowany typ obiektu języka *Python*, pozwalający w sposób leniwy generować kolejne elementy kolekcji.

Cykl życia modelu, będącego instancją jednej z powyższych klas, przedstawiono na rysunku 5.1.



Rysunek 5.1: Cykl życia modelu predykcyjnego w utworzonej implementacji

6 Weryfikacja eksperymentalna

W niniejszym rozdziale przedstawione zostały scenariusze oraz wyniki eksperymentów mających na celu doświadczalną weryfikację skuteczności metod opisywanych w poprzednich punktach.

6.1 Zbiory danych

6.1.1 *London01, London02, London03*

Wykorzystane dane pochodzą ze strony policji brytyjskiej¹⁷ i dotyczą przestępstw zarejestrowanych w Londynie na przestrzeni czasu od marca 2013 roku do marca 2014 roku. Każdy wiersz odpowiada jednemu odnotowanemu przestępstwu i wyposażony jest w *geotag*, zawierający długość i szerokość geograficzną miejsca zdarzenia, a także informacje o typie zajścia. W przeprowadzonych eksperymentach skupiono się na wykrywaniu konkretnego typu przestępstw, mianowicie przestępstw związanych z narkotykami. W tym celu przestępstwa te opatrzono etykietą klasy pozytywnej (1), a wszystkie pozostałe negatywnej (−1). Zbiór danych *London01* zawiera oryginalne dane przed transformacją, składające się z 6907 przykładów oraz 4 atrybutów:

- *lat* - szerokość geograficzna,
- *lon* - długość geograficzna,
- *month* - miesiąc zdarzenia,
- *class* - etykieta klasy.

Zbiory *London02* oraz *London03* zostały rozszerzone o dodatkowe cechy pochodne, opisane w dalszej części.

6.1.2 *California01, California02, California03*

Następna grupa danych dotyczy cen nieruchomości w stanie California [22], zebranych wraz z zestawem predyktorów ekonomicznych i przestrzennych.

¹⁷<http://data.police.uk/data/>

W przeprowadzonych eksperymentach ograniczono się do cech przestrzennych sformułowano i problem klasyfikacji binarnej, polegający na wykrywaniu najdroższych nieruchomości (o cenie wyższej niż 95-ty percentyl rozkładu). Nieruchomości te zostały opatrzone etykietą 1, a pozostałe -1 . Zbiór danych *California01* składa się z 20640 przykładów oraz 3 atrybutów:

- *lat* - szerokość geograficzna,
- *lon* - długość geograficzna,
- *class* - etykieta klasy.

Zbiory *California02* oraz *California03*, zostały rozszerzone o dodatkowe cechy pochodne, opisane w punkcie 6.2.1.

6.2 Cele i metody

Celem eksperymentów było wyznaczenie wpływu badanych metod na skuteczność klasyfikacji. Za główną metrykę skuteczności przyjęto miarę F_1 , ze względu na nierównomierną dystrybucję klas w wykorzystanych zbiorach danych. W celach porównawczych zbadano również *dokładność* klasyfikacji, rozumianą jako odsetek poprawnie zaklasyfikowanych przykładów. Powyższe metryki wyznaczono zarówno w funkcji liczby iteracji algorytmu, jak i wpływającego czasu. Wartości metryk dla danych treningowych obliczone zostały wprost, natomiast w celu oszacowania wartości tychże metryk dla danych testowych posłużono się 4-krotną walidacją krzyżową.

6.2.1 Inżynieria cech przestrzennych

Poza optymalizacją samych algorytmów i dostrajaniem ich parametrów, inżynieria cech jest jedną z najistotniejszych technik poprawy skuteczności modeli predykcyjnych. Technika ta polega na wyprowadzaniu cech pochodnych na podstawie tych już istniejących oraz zewnętrznych źródeł informacji, a także znajomości wiedzy dziedzinowej. W kontekście danych geograficznych, jednym z możliwych podejść jest obliczenie wartości predykatów przestrzennych, takich jak odległości czy warunki sąsiedztwa i potraktowanie rezultatów jako

nowe cechy. Bazując na zbiorach danych *London01* oraz *California01* wyznaczono następujące rodzaje cech pochodnych:

- **Liczba punktów w sąsiedztwie:** dla każdego z punktów wyznaczono dodatkowy atrybut będący liczbą punktów w zbiorze treningowym, oddalonych od danego punktu o nie więcej niż zadany promień. W tym celu wykorzystano predykat przestrzenny *ST_Distance_Sphere* systemu *PostGIS*, pozwalający na obliczanie odległości pomiędzy punktami na powierzchni Ziemi, a także operacje złączenia i agregacji tabel.
- **Odległość od punktów charakterystycznych:** dla każdego z punktów wyznaczono odległość od kilku wybranych punktów, które mogą być istotne w kontekście dziedziny problemu.
- **Odległość od otoczki wypukłej zbioru:** wyznaczono otoczkę wypukłą zbioru treningowego, będącą najmniejszym wielokątem wypukłym zawierającym wszystkie jego punkty, a następnie obliczono odległość każdego z punktów od otoczki.

Wykorzystując powyższe metody, wyprowadzono następujące atrybuty pochodne:

- **London02**
 - *N_IN_NEIGHBORHOOD* - liczba przestępstw w zbiorze treningowym w odległości mniejszej niż 500 metrów od danego,
- **London03**
 - *D_TRAFALGAR*, *D_ISLINGTON*, *D_LAMBETH* - odległość od trzech punktów charakterystycznych, będących centrami obszarów kojarzonych z wysoką przestępczością,
- **California02**
 - *DIST_FROM_BOUNDARY* - odległość nieruchomości od otoczki wypukłej zbioru trenującego. Motywacją dla zbadania tego atrybutu było dodanie informacji na temat odległości budynków od wybrzeża oraz granic stanu,

- **California03**

- *DIST_FROM_COAST* - odległość od przecięcia otoczki wypukłej z prostokątem obejmującym nabrzeże. Celem tej operacji było usunięcie z rozważań w poprzednim punkcie granic stanu, a zbadanie jedynie odległości od nabrzeża.

6.2.2 Lokalne warianty boostingu

Do wszystkich wariantów zbiorów danych otrzymanych przy pomocy inżynierii cech zastosowano zarówno referencyjny algorytm *AdaBoost*, jak i jego wariant lokalny (nazwany *LocalBoost*), polegający na wprowadzeniu pomocniczych klasyfikatorów (zdefiniowanych w punkcie 4.2.1). Etapem wstępnym eksperymentów było porównanie funkcji łączących. Najlepszą krzywą uczenia uzyskano przy pomocy nieliniowej funkcji łączącej dla parametru $\gamma \approx 0.25$. Dla takiego zestawu parametrów przeprowadzono dalsze, szczegółowe eksperymenty.

6.3 Scenariusze eksperymentów

Szczegółowe scenariusze pomiarów zawarte zostały w pliku źródłowym *benchmark_configs.py*. Dla każdego scenariusza określone zostały następujące parametry:

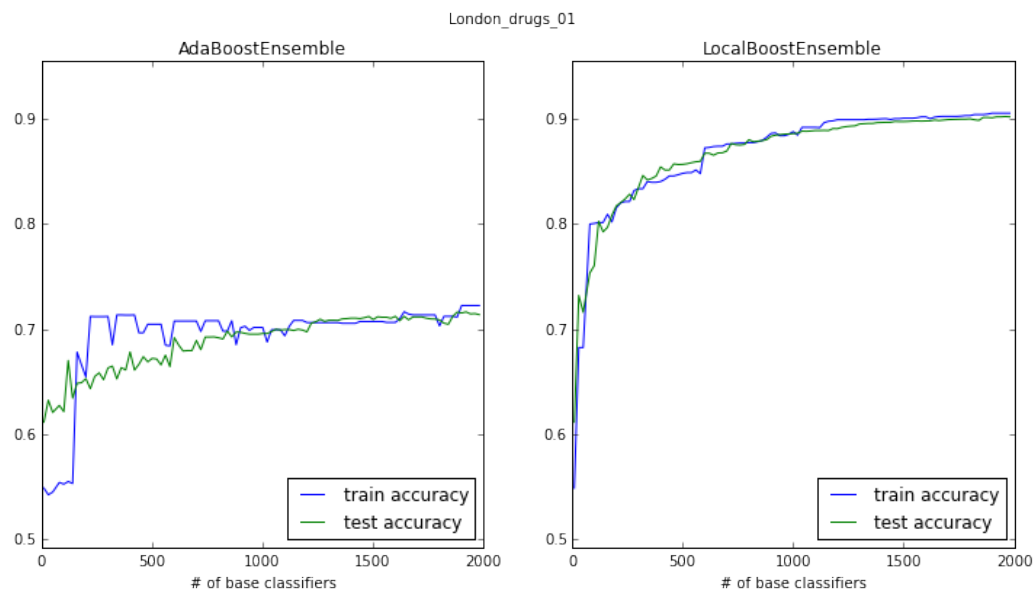
- źródłowy zbiór danych,
- lista atrybutów pełniących rolę predyktorów oraz atrybut zawierający etykiety klas,
- badany algorytm oraz algorytm referencyjny,
- dodatkowe parametry dla każdego z powyższych,
- wektor iteracji, określający zakres oraz granularność wykonywanych pomiarów,
- krotność walidacji krzyżowej.

Każdy ze źródłowych zbiorów danych zawarty został w co najmniej jednym scenariuszu, w którym jego opisane wyżej cechy pierwotne i pochodne zostały użyte jako predyktory, a etykieta klasy jako atrybut docelowy klasyfikacji binarnej. Dla każdego scenariusza wyznaczono krzywą uczenia, będącą zapisem wartości metryk (dokładności oraz miary F_1) w funkcji liczby iteracji oraz czasu. Przyjęto zakres od 1 do 2000 iteracji z odstępami 20 iteracji pomiędzy kolejnymi punktami pomiarowymi. Krzywe uczenia wyznaczono zarówno dla błędów treningowych jak i testowych, a także w funkcji czasu.

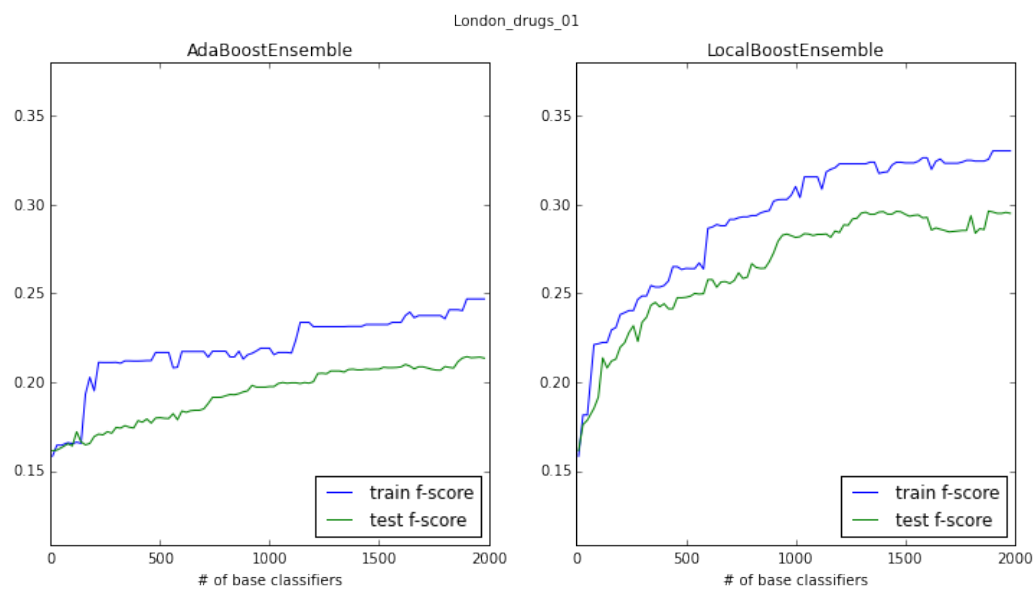
6.4 Wyniki eksperymentów

Wyniki eksperymentów przedstawione zostały przy użyciu dwóch typów wizualizacji: typ pierwszy zawiera zestawienie dwóch wykresów, z których lewy odpowiada bazowej wersji algorytmu *AdaBoost* natomiast prawy wersji zmodyfikowanej. Na każdym z nich przedstawiona została jedna z metryk (dokładność lub miara F_1) dla danych treningowych oraz testowych w funkcji liczby iteracji. Osie wykresów zostały identycznie wyskalowane. Kolejne rysunki od Rys. 6.1 do Rys. 6.12 ilustrują wyniki uzyskane dla następujących zbiorów: *London01*, *London02*, *London03*, *California01*, *California02*, *California03*.

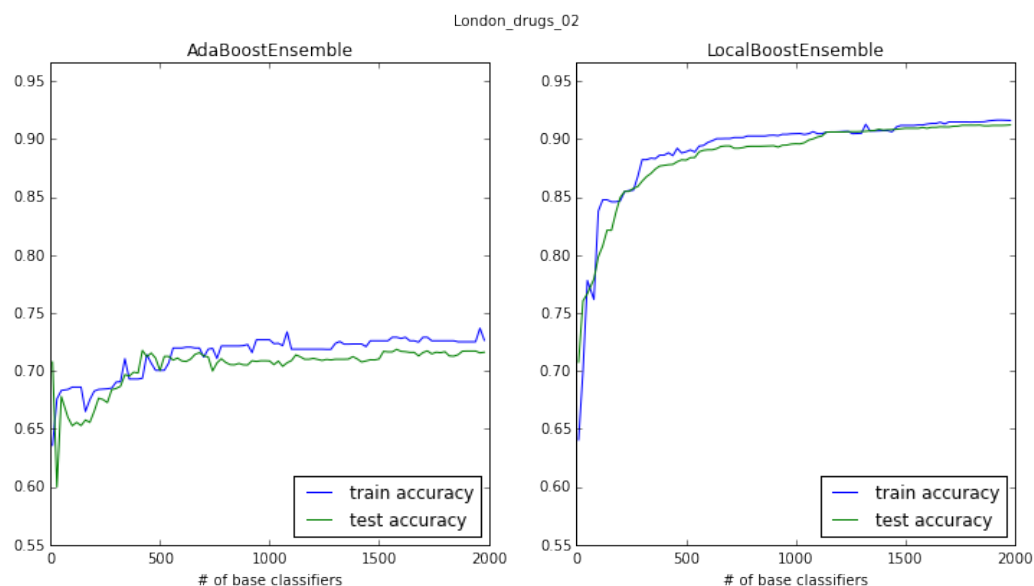
Z kolei drugi typ wizualizacji zawiera zestawienie metryk dla obu wariantów algorytmu na jednym wykresie w funkcji czasu. Celem takiego przedstawienia wyników jest unaocznienie jaką dokładność klasyfikacji pozwala uzyskać każdy wariant po upływie tej samej liczby sekund. Wyniki obrazujące zależności czasowe dla wymienionych wcześniej zbiorów przedstawione są na kolejnych rysunkach od Rys. 6.13 do Rys. 6.24.



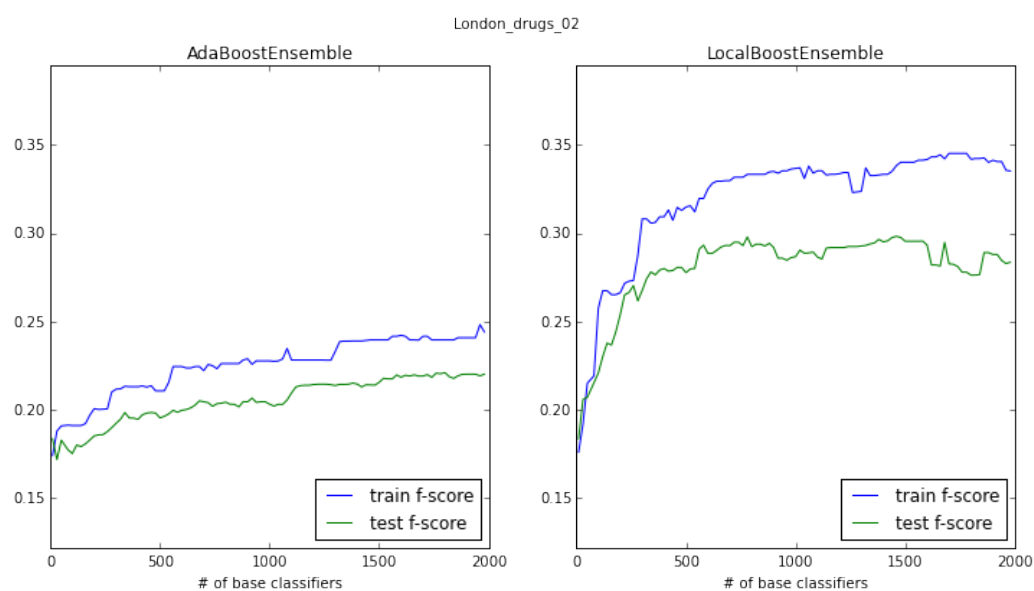
Rysunek 6.1: Dokładność względem liczby iteracji na zbiorze *London01*



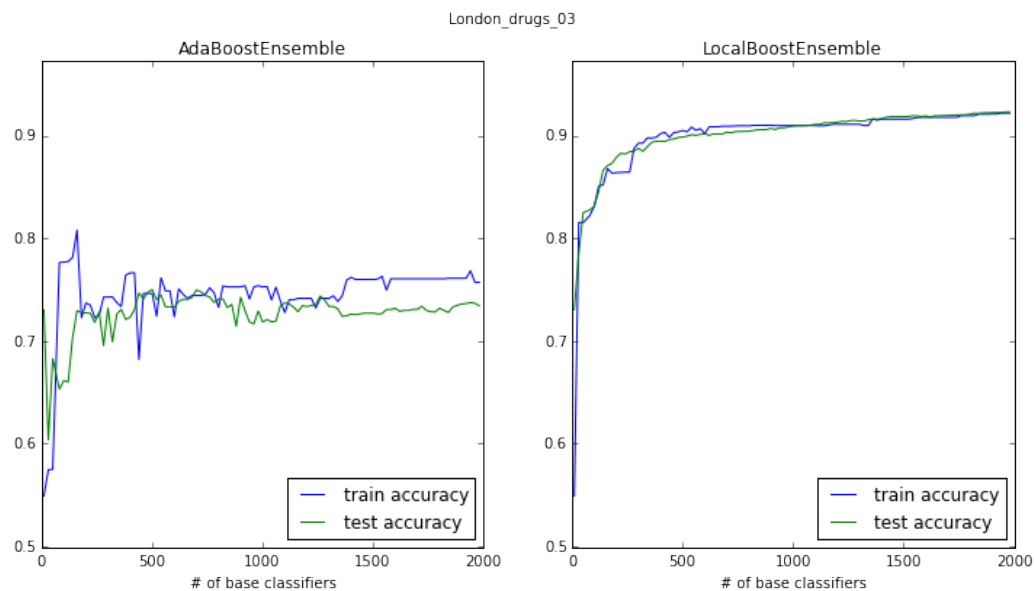
Rysunek 6.2: Miara F_1 względem liczby iteracji na zbiorze *London01*



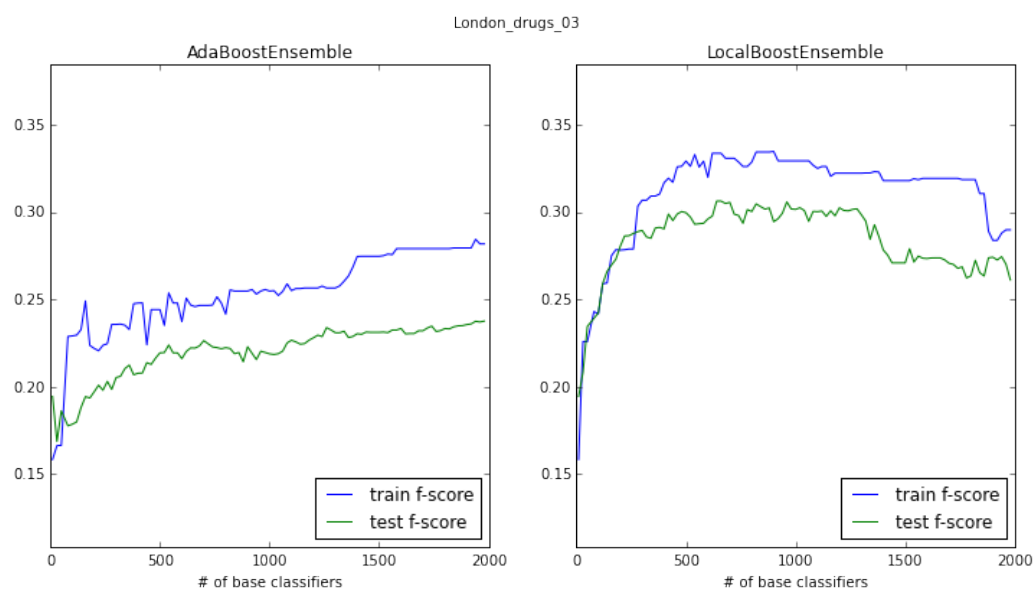
Rysunek 6.3: Dokładność względem liczby iteracji na zbiorze *London02*



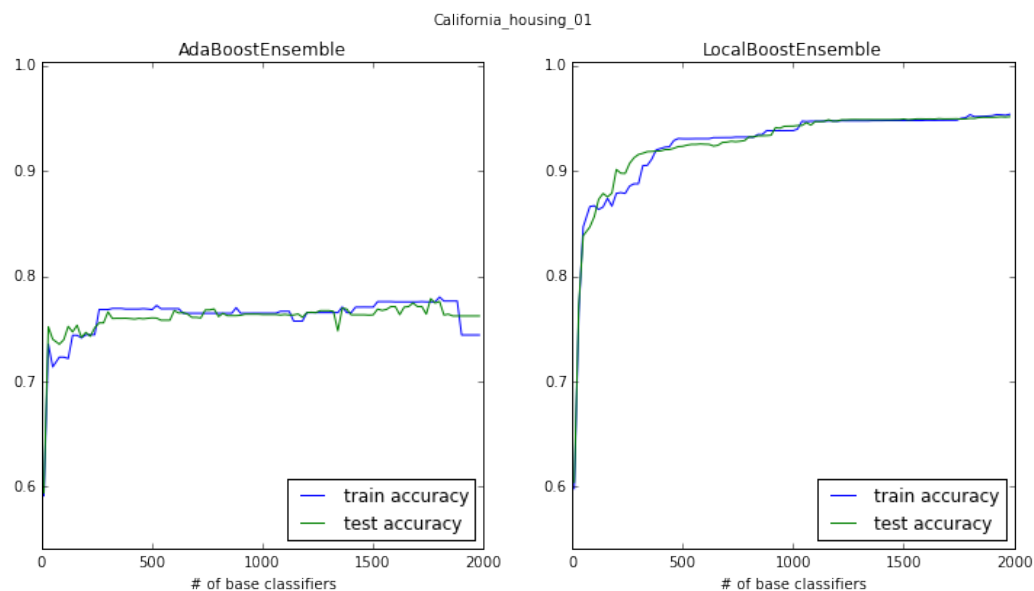
Rysunek 6.4: Miara F_1 względem liczby iteracji na zbiorze *London02*



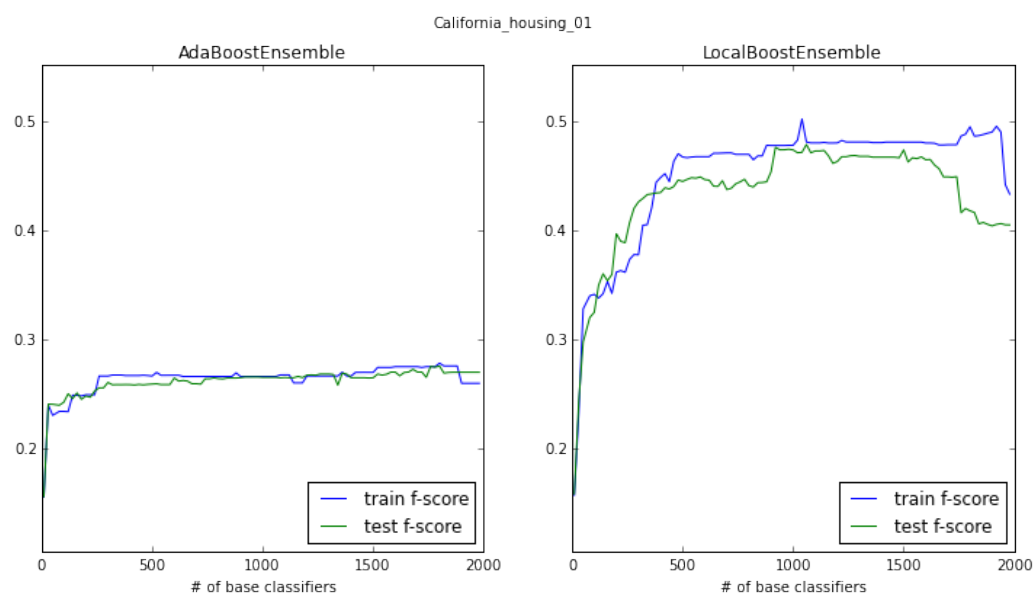
Rysunek 6.5: Dokładność względem liczby iteracji na zbiorze *London03*



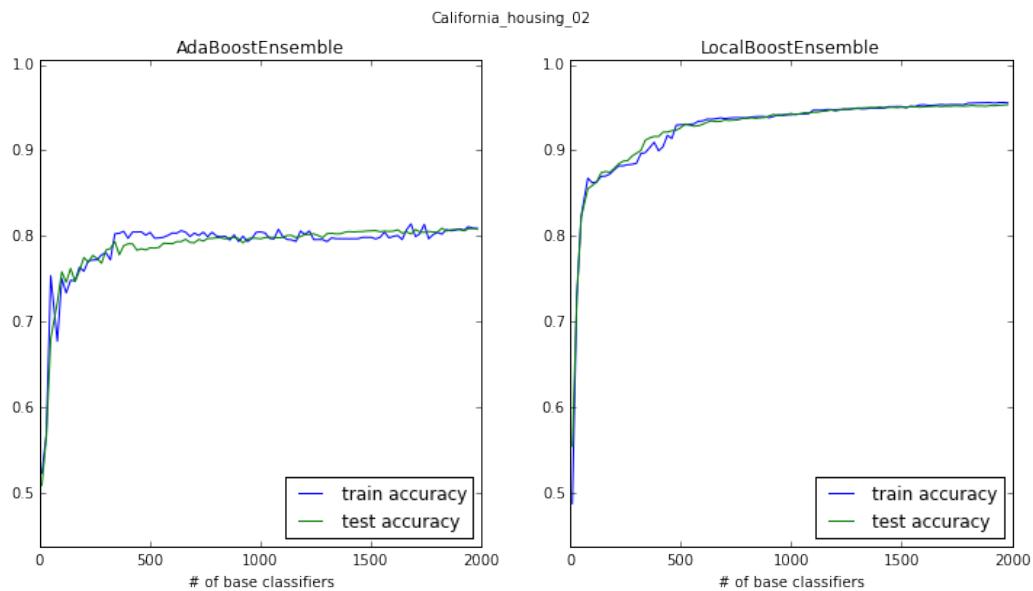
Rysunek 6.6: Miara F_1 względem liczby iteracji na zbiorze *London03*



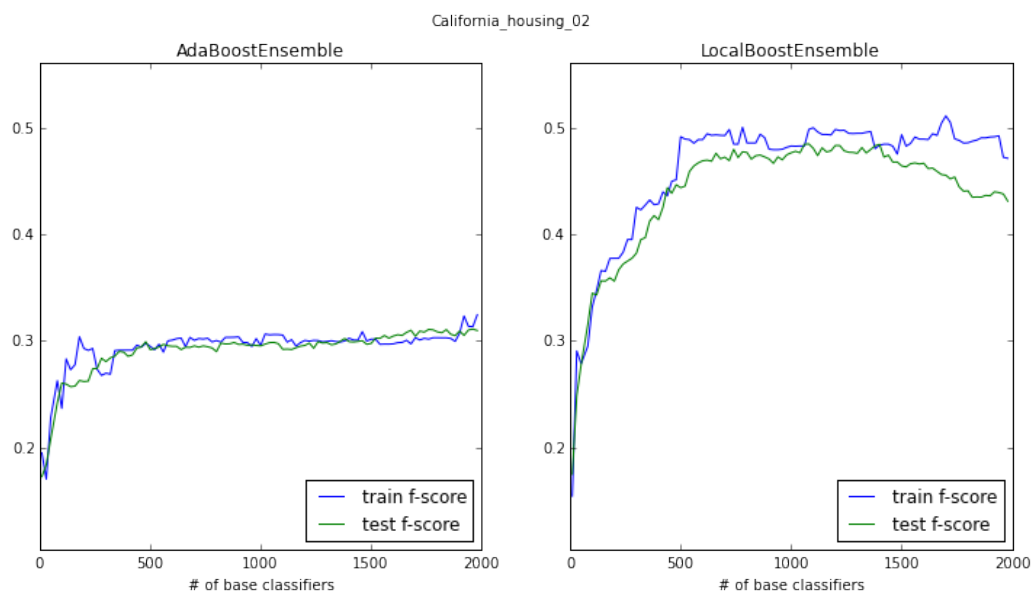
Rysunek 6.7: Dokładność względem liczby iteracji na zbiorze *California01*



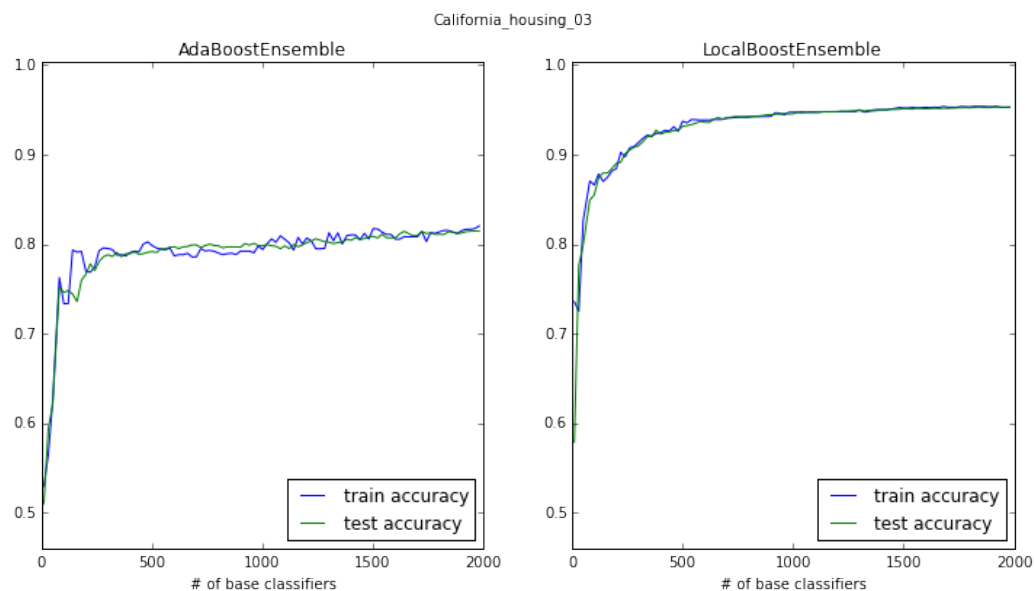
Rysunek 6.8: Miara F_1 względem liczby iteracji na zbiorze *California01*



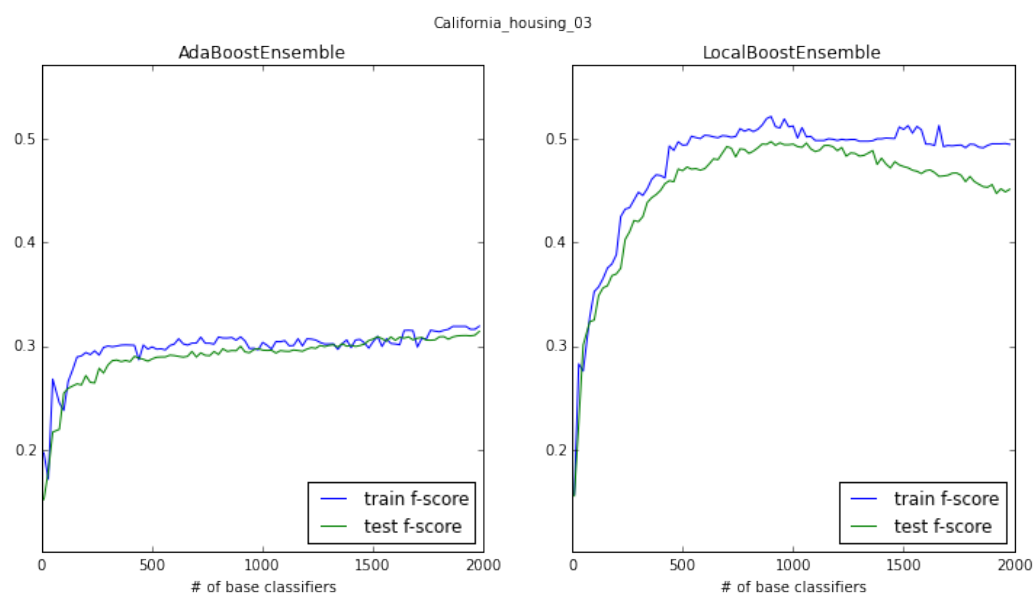
Rysunek 6.9: Dokładność względem liczby iteracji na zbiorze *California02*



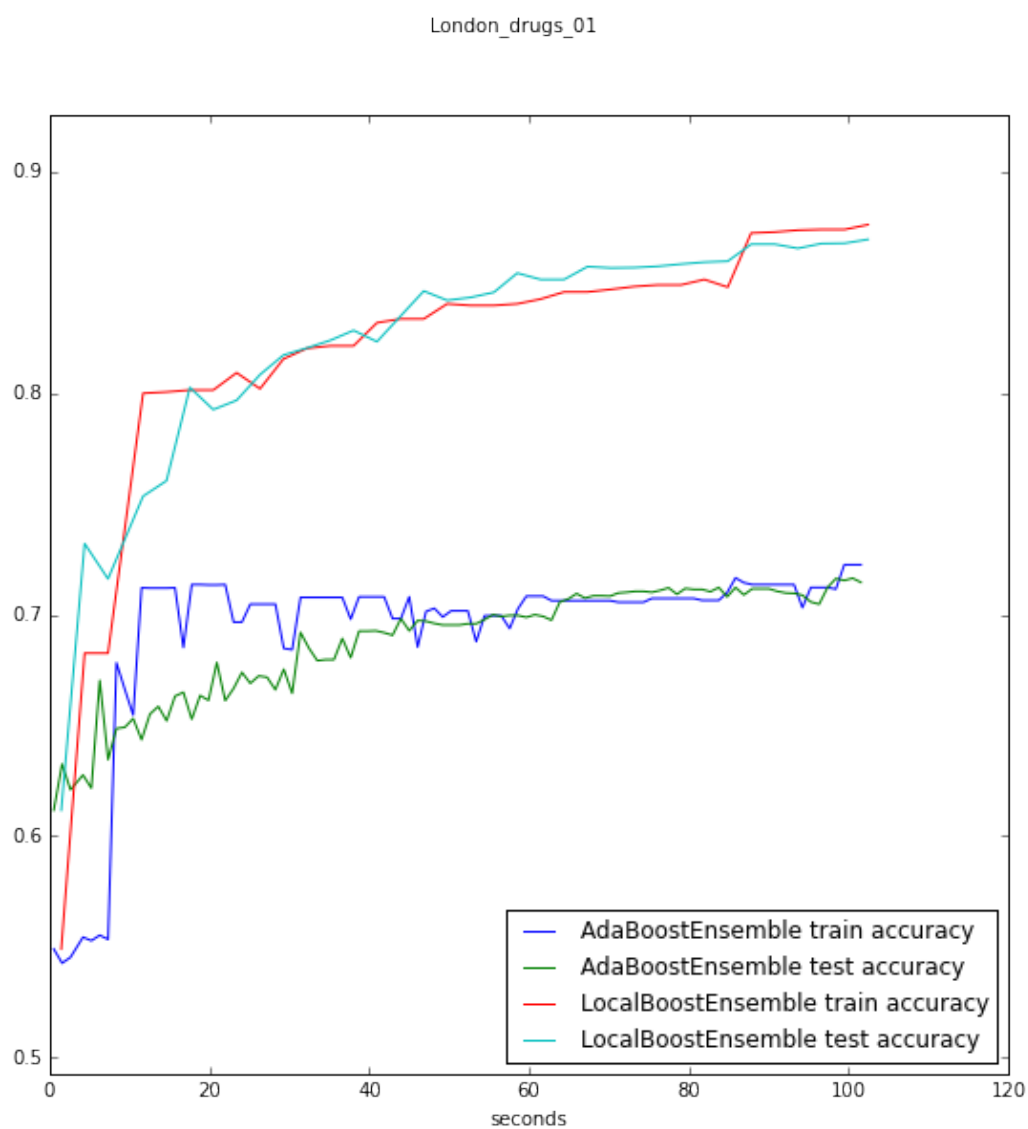
Rysunek 6.10: Miara F_1 względem liczby iteracji na zbiorze *California02*



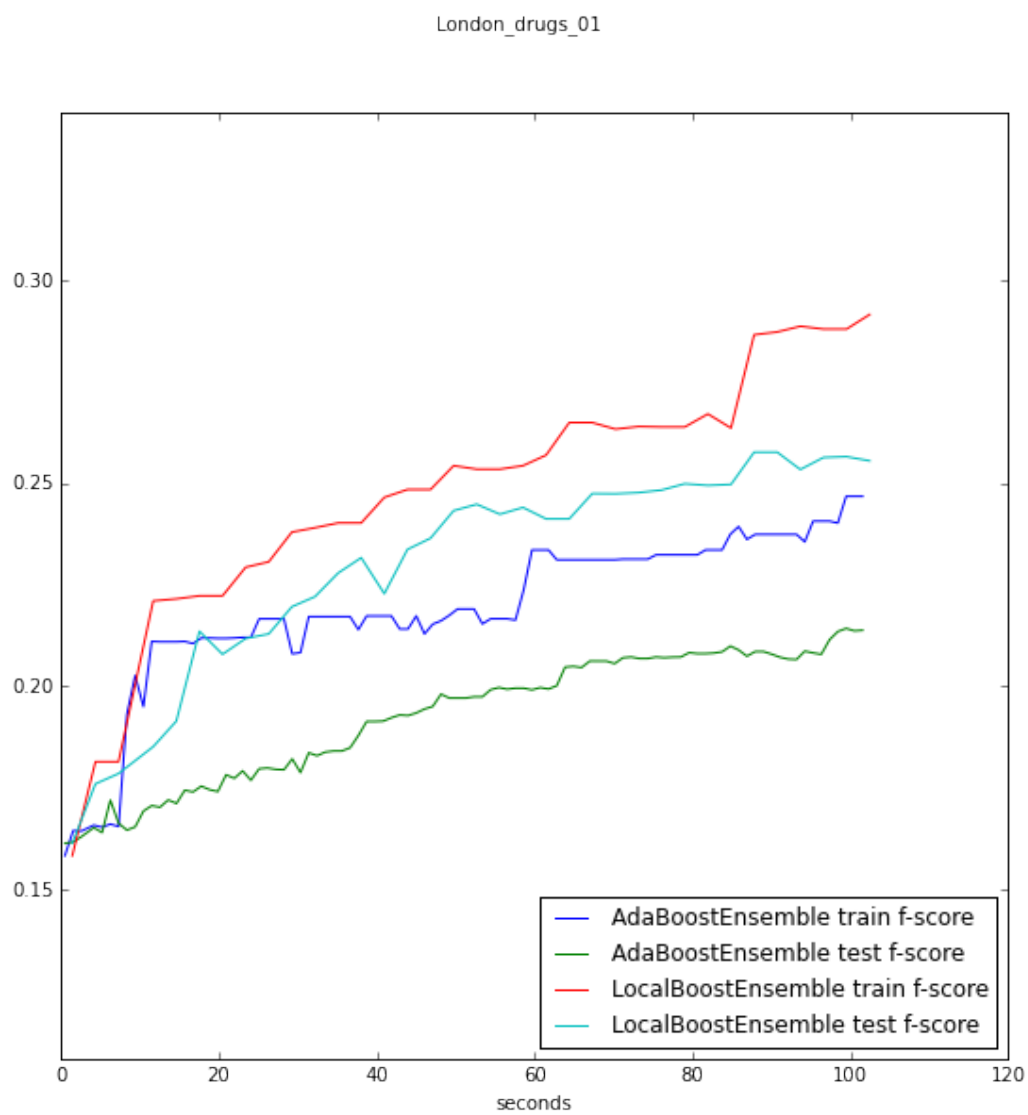
Rysunek 6.11: Dokładność względem liczby iteracji na zbiorze *California03*



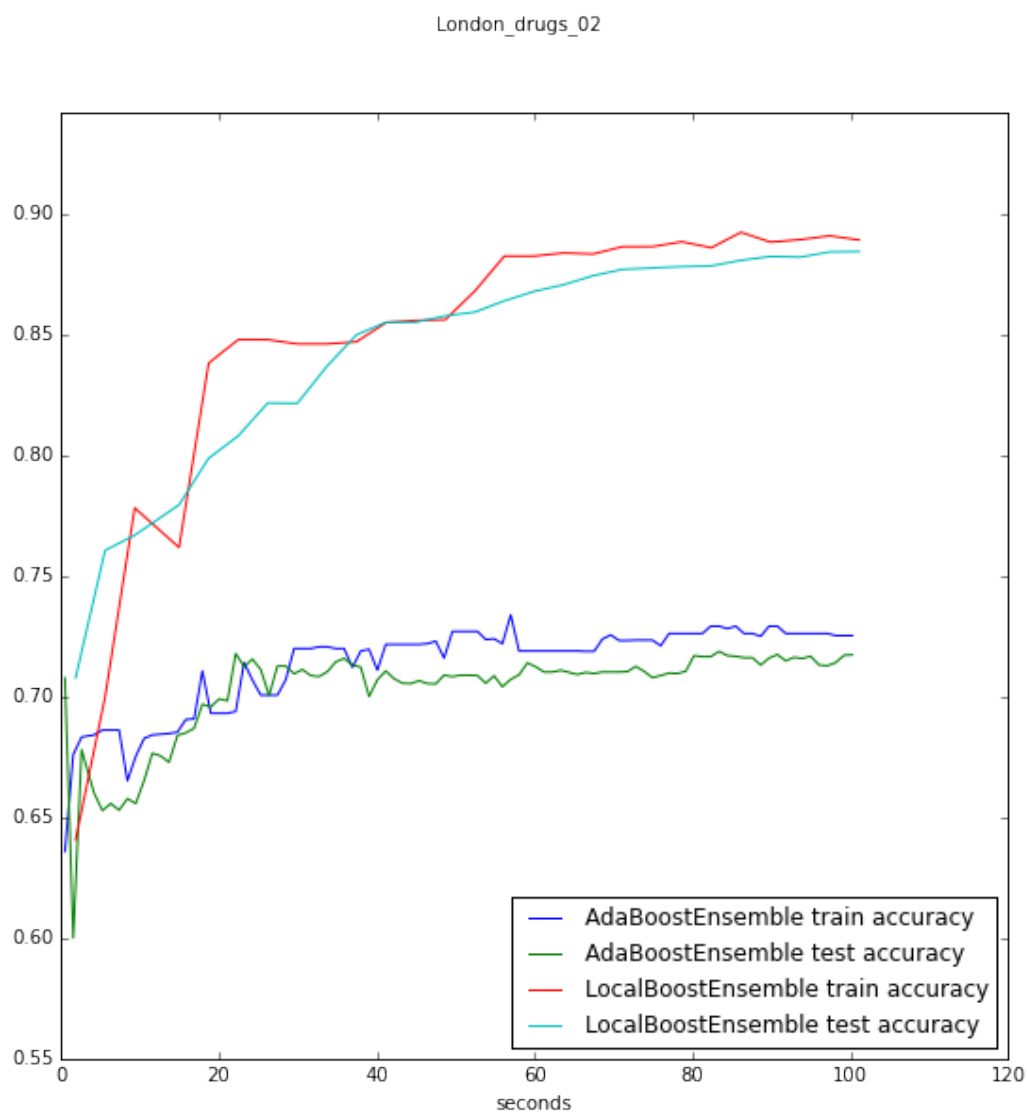
Rysunek 6.12: Miara F_1 względem liczby iteracji na zbiorze *California03*



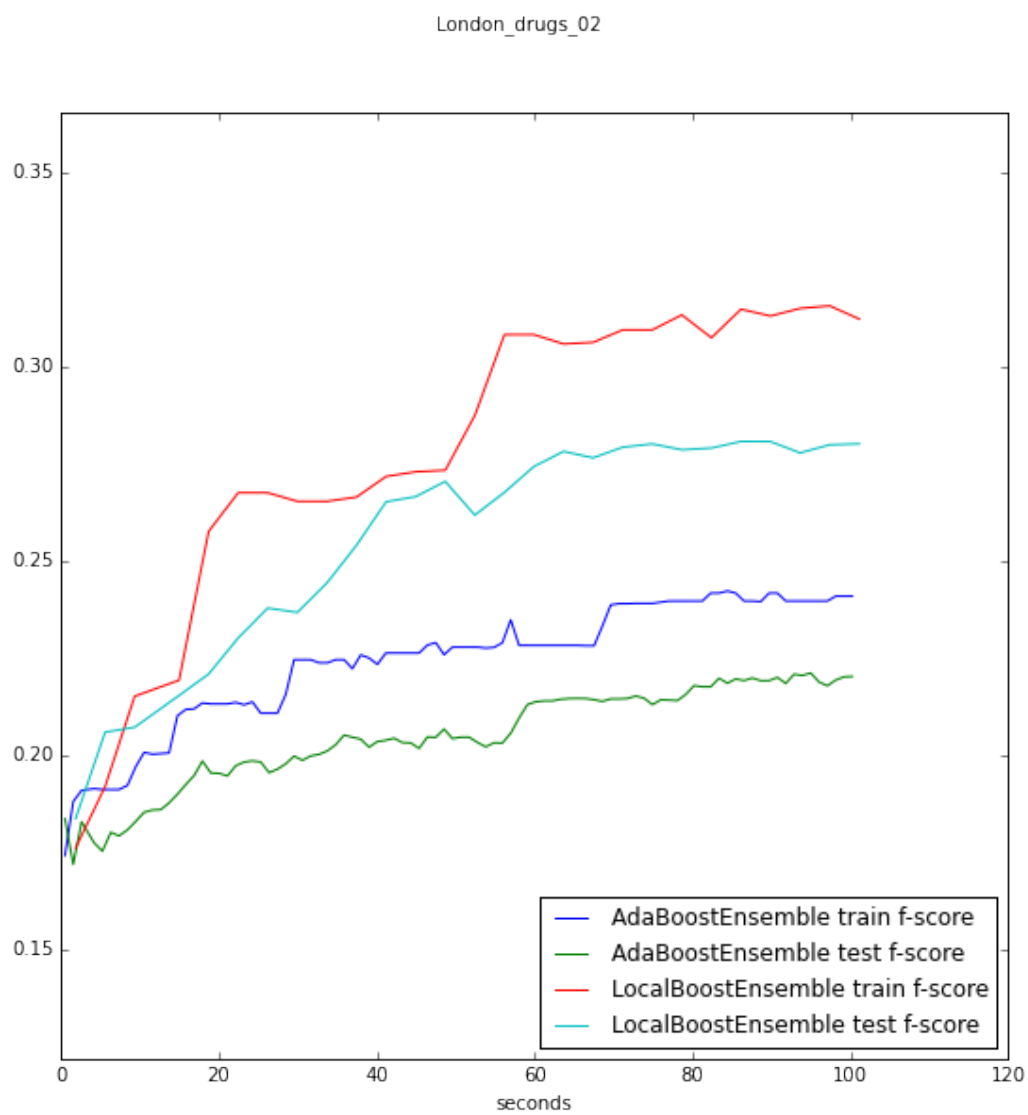
Rysunek 6.13: Dokładność względem czasu na zbiorze *London01*



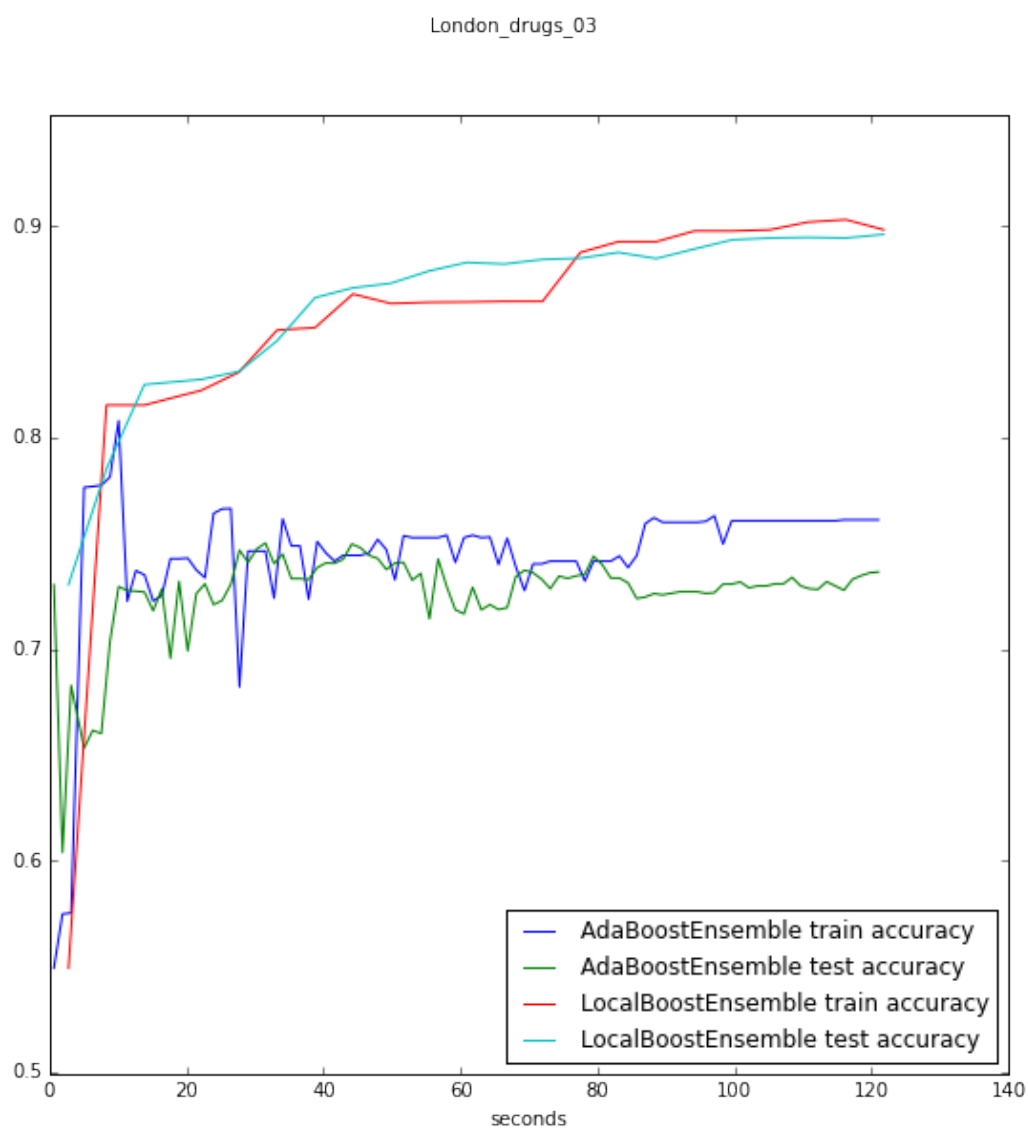
Rysunek 6.14: Miara F_1 względem czasu na zbiorze *London01*



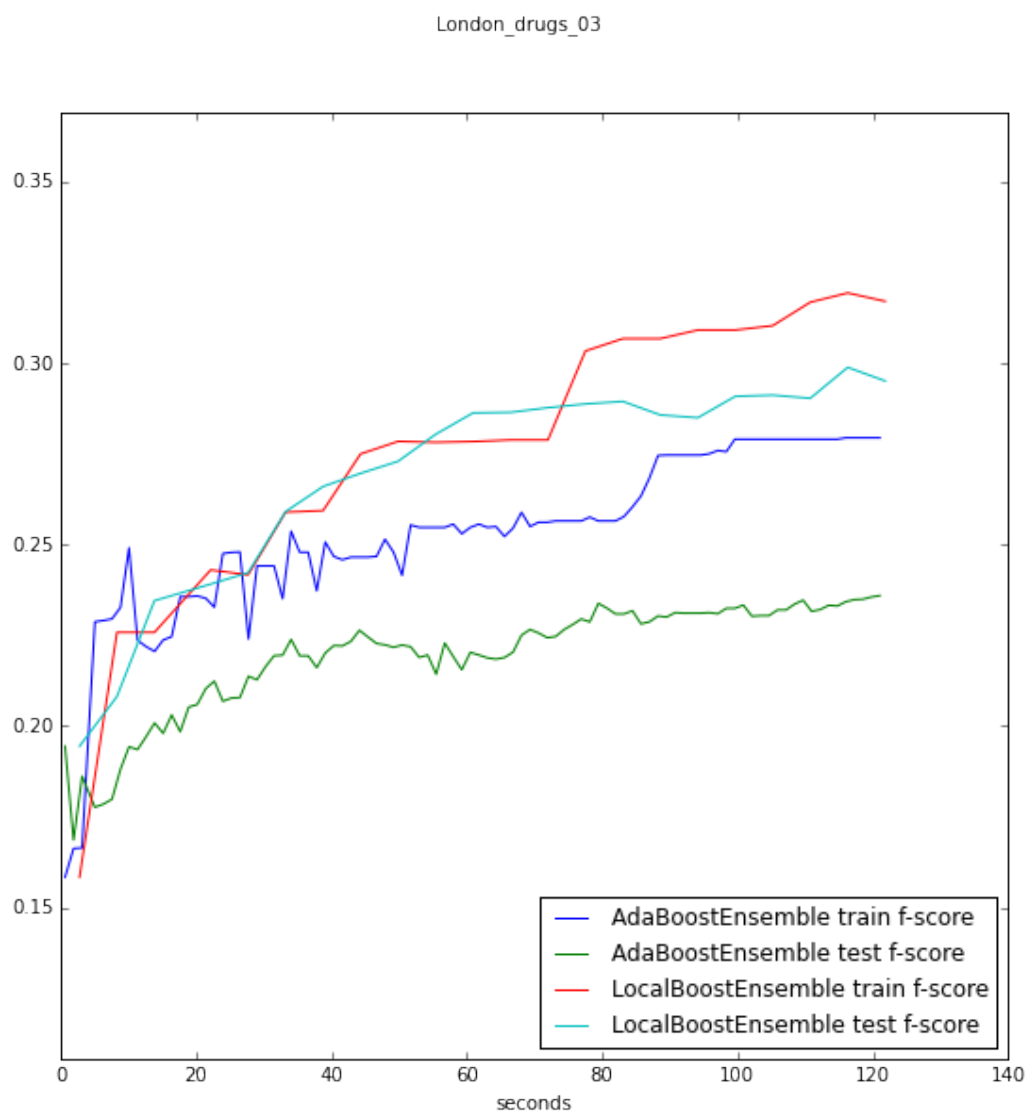
Rysunek 6.15: Dokładność względem czasu na zbiorze *London02*



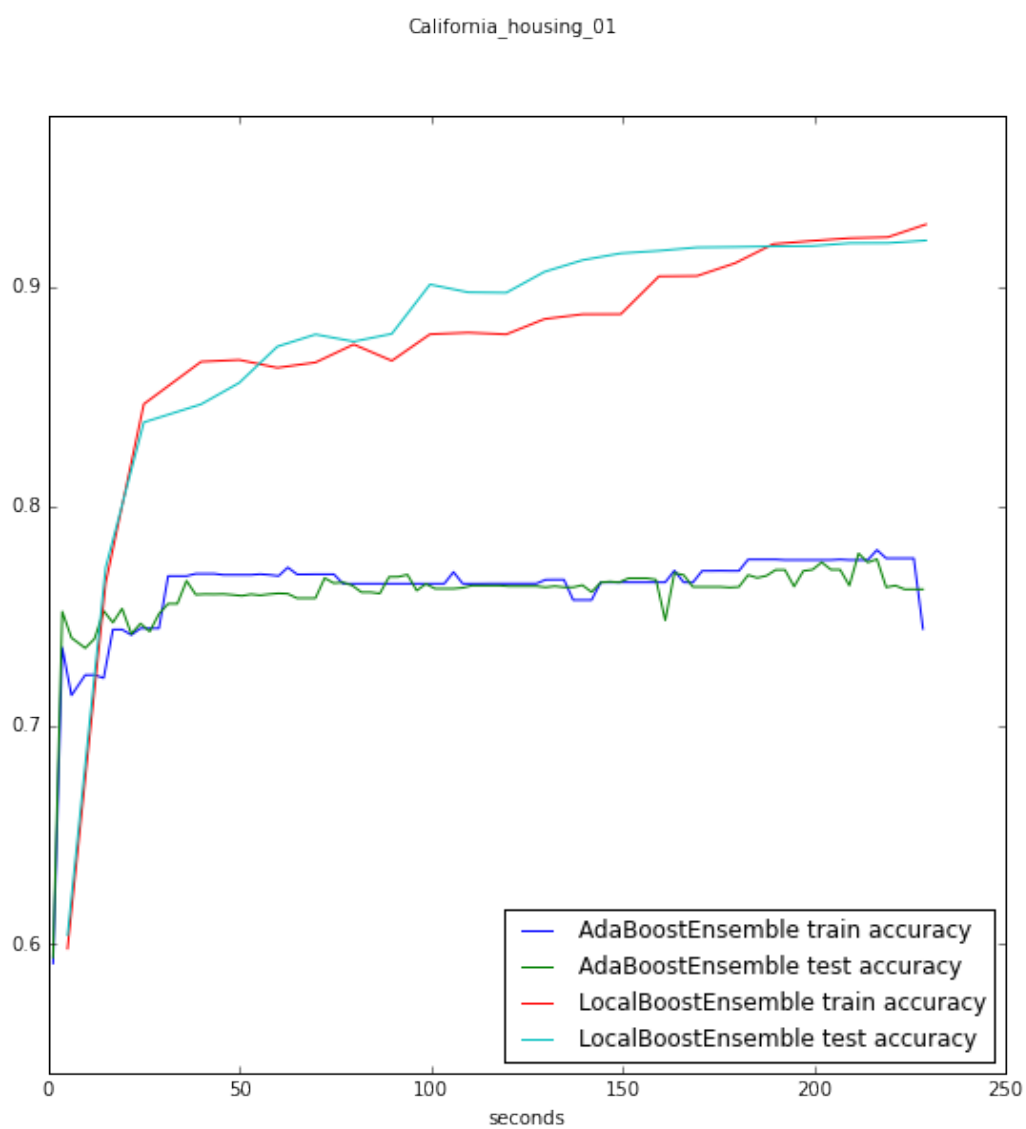
Rysunek 6.16: Miara F_1 względem czasu na zbiorze *London02*



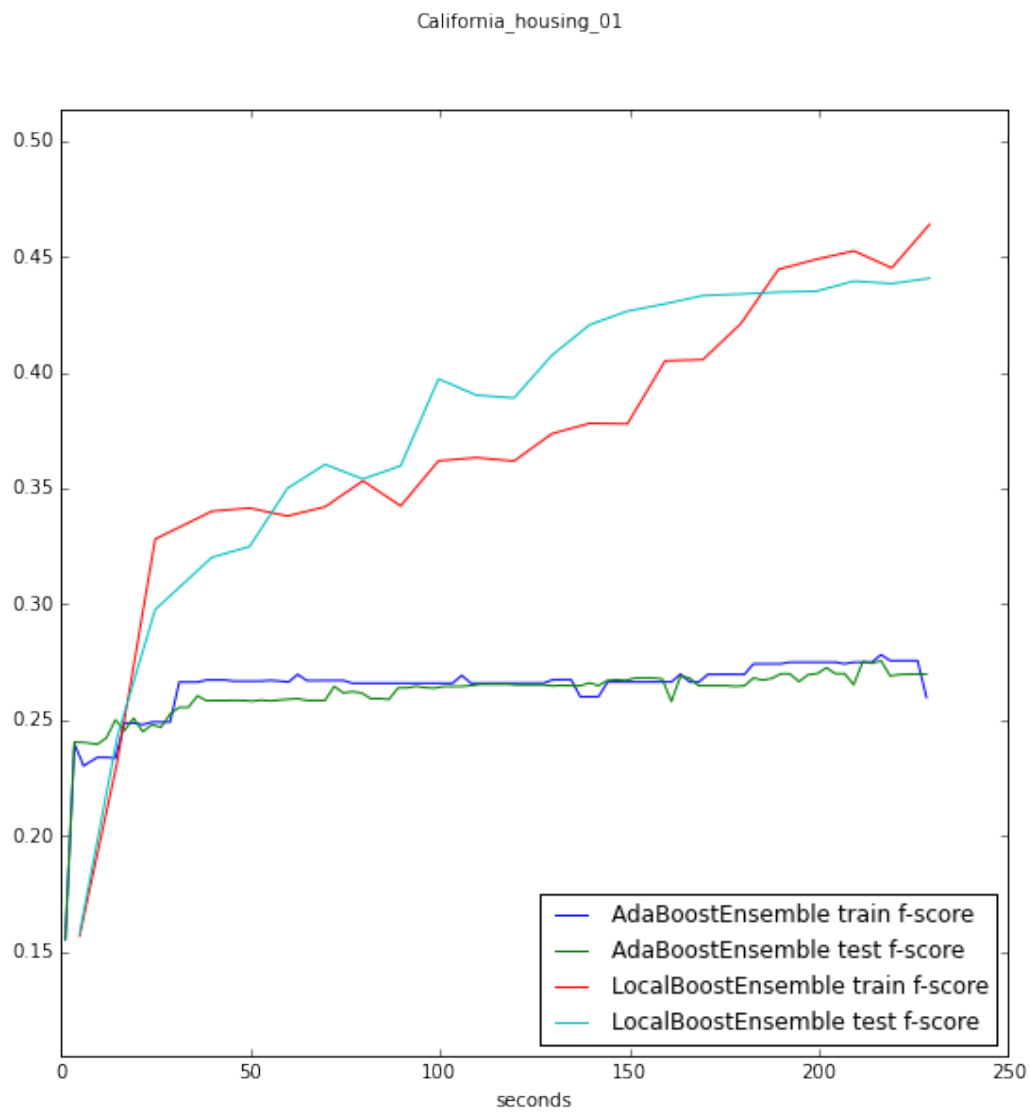
Rysunek 6.17: Dokładność względem czasu na zbiorze *London03*



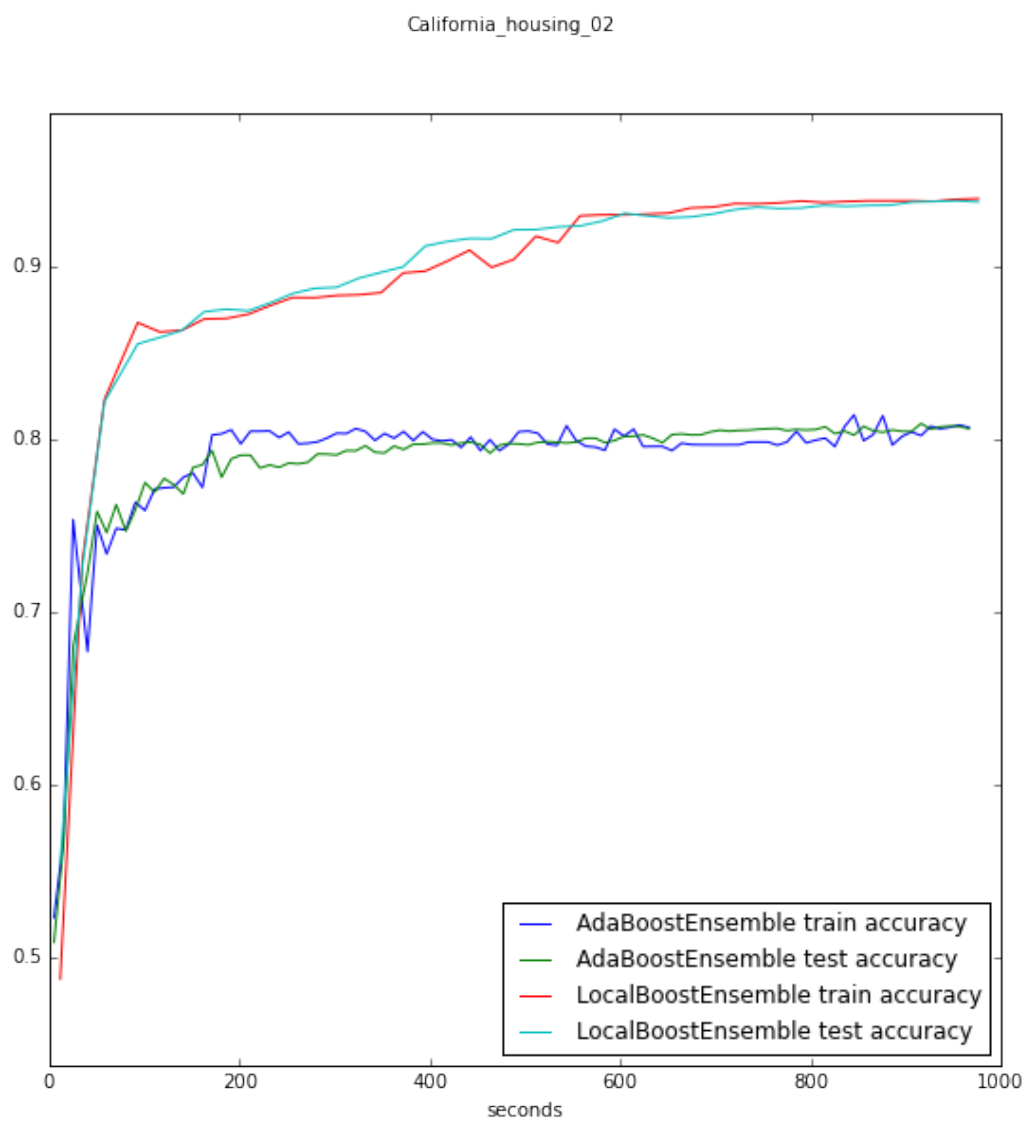
Rysunek 6.18: Miara F_1 względem czasu na zbiorze *London03*



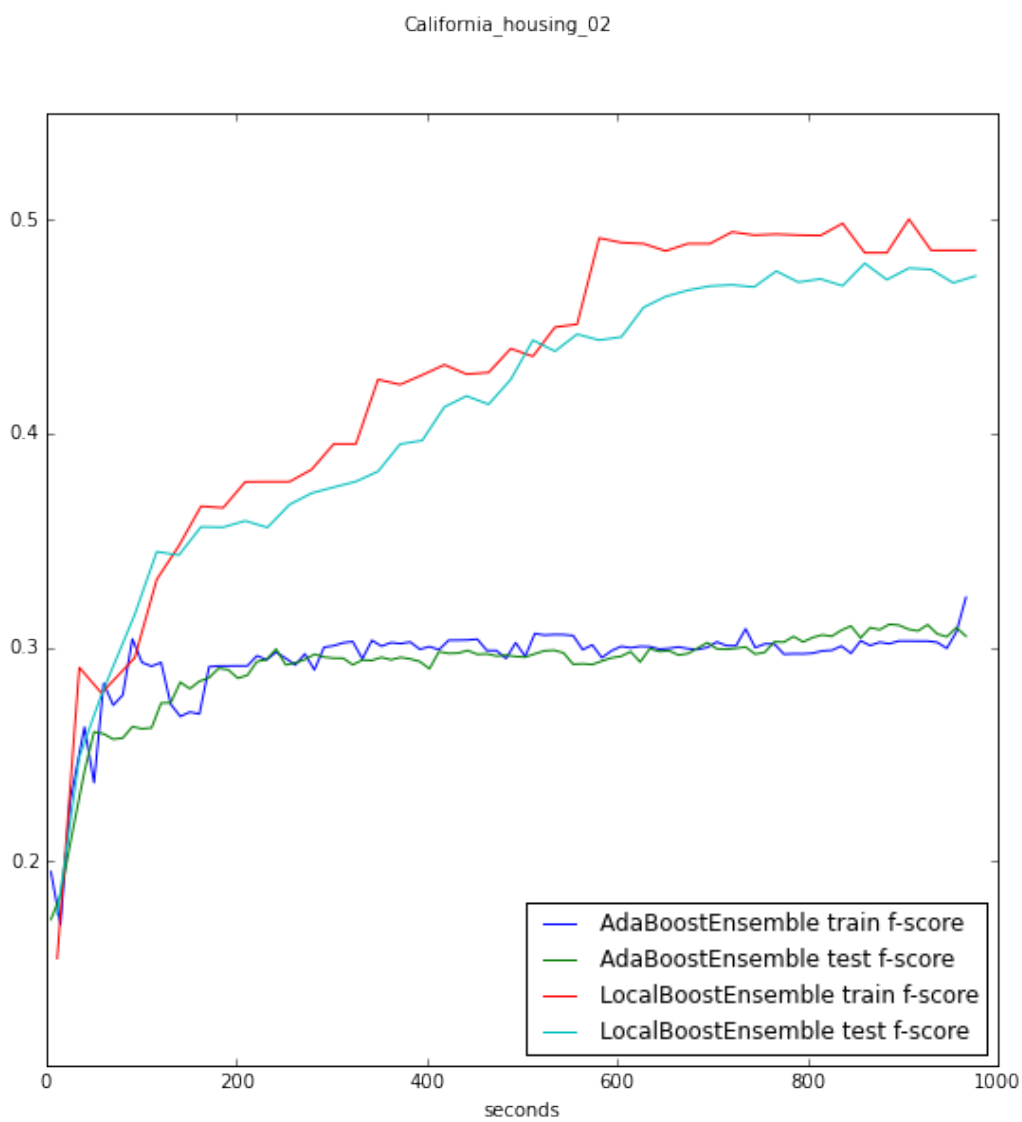
Rysunek 6.19: Dokładność względem czasu na zbiorze *California01*



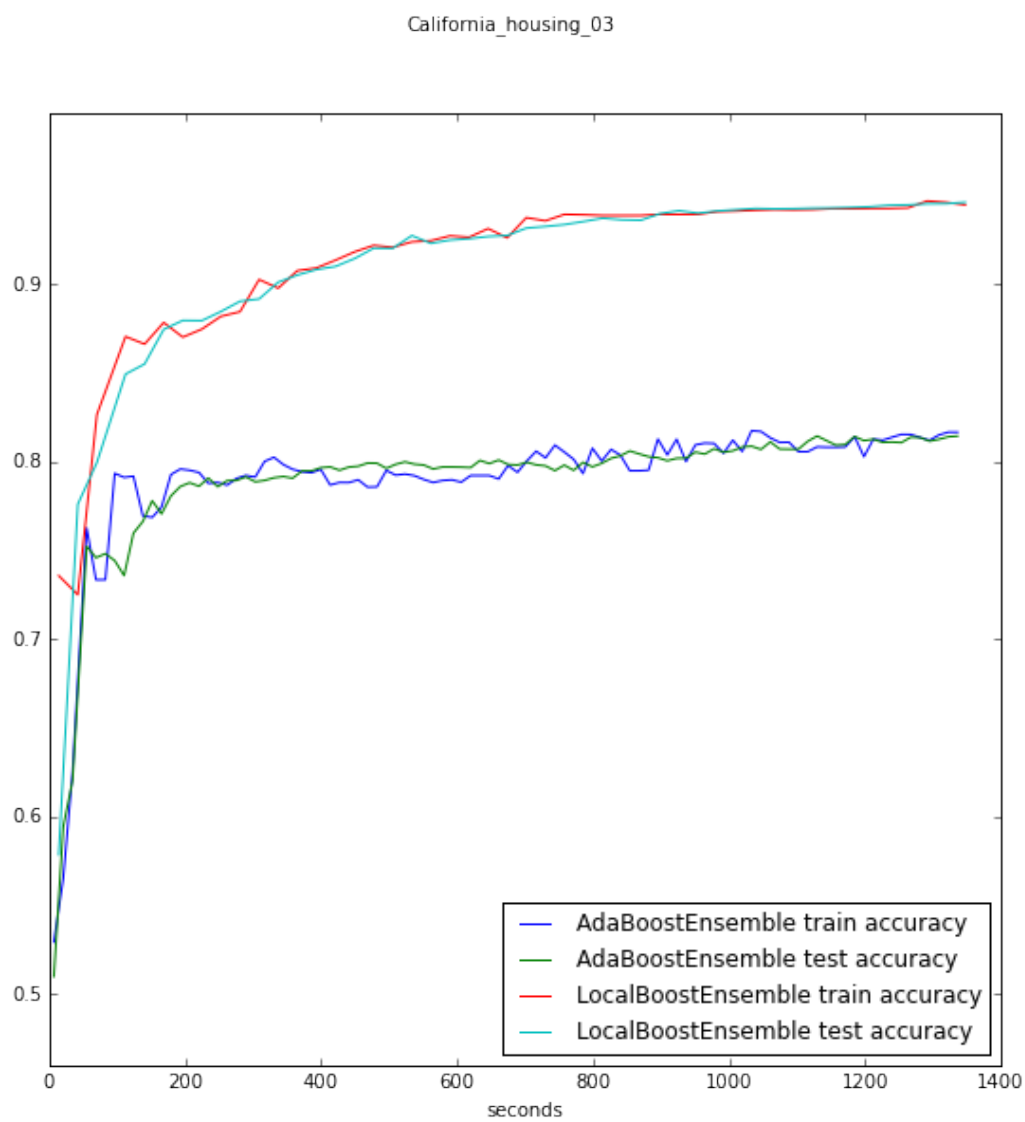
Rysunek 6.20: Miara F_1 względem czasu na zbiorze *California01*



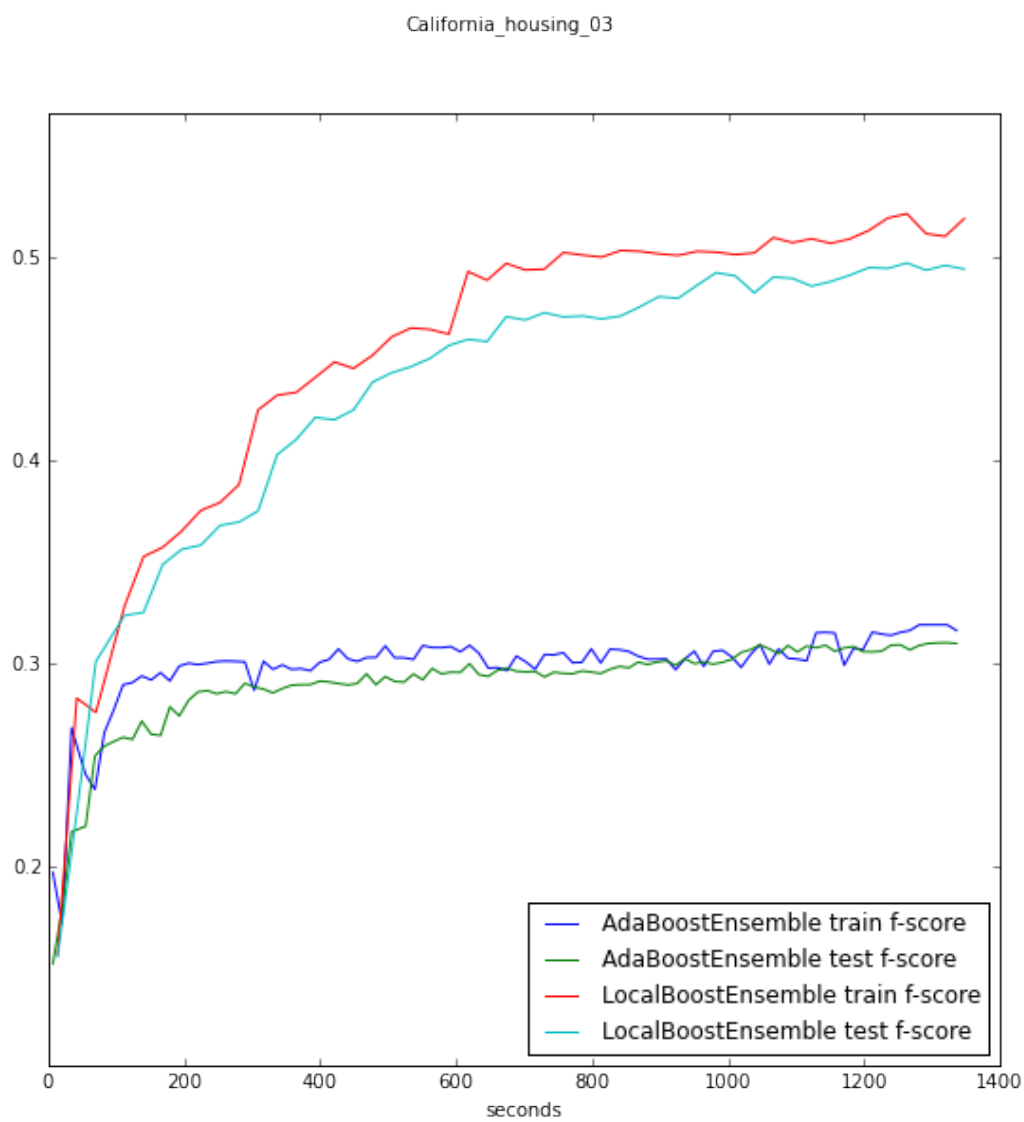
Rysunek 6.21: Dokładność względem czasu na zbiorze *California02*



Rysunek 6.22: Miara F_1 względem czasu na zbiorze *California02*



Rysunek 6.23: Dokładność względem czasu na zbiorze *California03*



Rysunek 6.24: Miara F_1 względem czasu na zbiorze *California03*

6.5 Analiza wyników

Uzyskane rezultaty sugerują poprawę działania algorytmu *AdaBoost* w wariacie zmodyfikowanym o lokalne wagi klasyfikatorów bazowych względem wariantu bazowego. Poprawa jest widoczna zarówno we wzroście dokładności, jak i miary F_1 , a jej wymiar jest zależny od użytego zbioru danych. Obie badane metryki osiągają wyższe wartości po tej samej liczbie iteracji dla boostingu lokalnego, większa jest także ich maksymalna wartość osiągana w procesie uczenia. Wzrost maksymalnej wartości metryk zawiera się w przedziale 10% – 30% w przypadku dokładności oraz 12% – 45% w przypadku miary F_1 . Najwyższą względną poprawę uzyskano dla zbiorów danych niewzbogaconych o dodatkowe cechy pochodne, w szczególności dla zbioru *London01*.

Zestawienie obu wariantów algorytmu w funkcji czasu również wskazuje na lepsze wyniki uzyskane dla algorytmu zmodyfikowanego. Różnica jest szczególnie wyraźna w przypadku dokładności - jej wartość stabilizuje się po około minucie na poziomie o 12%–20% większym. Dalszy wzrost dokładności następuje w obu wariantach bardzo powoli. Zachowanie miary F_1 w funkcji czasu wykazuje bardziej stopniowy charakter, jednak również z korzyścią dla wariantu lokalnego.

Ogólnym spostrzeżeniem płynącym z analizy uzyskanych wyników jest fakt szybkiej stabilizacji dokładności oraz miary F_1 algorytmu *AdaBoost* na takim poziomie, po którego osiągnięciu dalszy wzrost jest bardzo powolny. Fakt ten wynikać może ze słabej separacji klas w użytych zbiorach danych oraz obecności szumu, na który algorytm *AdaBoost* jest wrażliwy. Wrażliwość ta przejawia się w silnym zawyżaniu wag przykładów zaszumionych i w konsekwencji pogorszeniu jakości modelu. Wariant lokalny zdaje się być w mniejszym stopniu zależnym od tego problemu.

Dodatkową obserwacją jest stopniowy spadek wartości miary F_1 uzyskanej przez lokalny klasyfikator, rozpoczynający się po przekroczeniu pewnej liczby iteracji. Spadek ten widoczny jest na wykresach z Rys. 6.6 oraz Rys. 6.8 po około 1500 iteracjach i dotyczy zarówno miary treningowej jak i testowej. Przyczyną tego zjawiska może być kumulacja błędów klasyfikatorów pomoc-

nicznych, które coraz gorzej są w stanie przewidywać obszar kompetencji klasyfikatorów bazowych i w efekcie zaburzają ostateczną predykcję.

Cechy pochodne wprowadzone do zbiorów danych *London02* oraz *London03* spowodowały niewielką poprawę dokładności oraz miary F_1 dla obu wariantów klasyfikatorów, jednak poprawa widoczna jest głównie w metrykach treningowych. Przebiegi metryk testowych poprawiły się w mniejszym stopniu, a niekiedy nawet uległy pogorszeniu, co wskazywać może na wystąpienie problemu nadmiernego dopasowania.

Należy zwrócić uwagę, że dodatkowe cechy wprowadzone do zbiorów *California02* oraz *California03* wywarły bardzo niewielki wpływ na kształt krzywych uczenia. Pewien wzrost wartości metryk po 2000 iteracji zaobserwowano dla bazowego wariantu algorytmu *AdaBoost*. Wzrost ten wyniósł około 3% w przypadku dokładności oraz około 5% w przypadku miary F_1 . Z kolei różnica w metodach inżynierii cech pomiędzy zbiorami *California02* i *California03* nie miała zauważalnego wpływu na przebieg krzywych uczenia. Nie zaobserwowano również istotnej poprawy jakości klasyfikatorów lokalnych po wprowadzeniu dodatkowych cech.

7 Podsumowanie pracy

7.1 Wnioski końcowe

Utworzone w niniejszej pracy oprogramowanie, realizujące grupowy algorytm klasyfikacji *AdaBoost* oraz jego zmodyfikowany wariant lokalny, zostało wykorzystane do eksperymentalnego zweryfikowania skuteczności proponowanych modyfikacji. Eksperymenty przeprowadzono w oparciu o geograficzne zbiory danych, w różnym stopniu rozbudowane przy pomocy metod inżynierii cech przestrzennych. Rezultaty przeprowadzonych eksperymentów wskazują na poprawę skuteczności klasyfikatorów opartych na koncepcji boostingu lokalnego zarówno w funkcji liczby iteracji, jak i czasu. Wprowadzone cechy pochodne również wpłynęły na poprawę dokładności klasyfikacji, jednak w mniejszym stopniu i w niektórych przypadkach doprowadziły do wystąpienia nadmiernego dopasowania. Uzyskane empirycznie zależności wykazują pewne cechy, które wymagają dodatkowych eksperymentów oraz analiz. Przykładem może być stopniowy spadek miary F_1 klasyfikatorów lokalnych po przekroczeniu pewnej liczby iteracji. Ze względu na modularny charakter utworzonego oprogramowania oraz zgodność z konwencjami pakietu analitycznego *scikit-learn*, możliwa jest w przyszłości łatwa rozbudowa bibliotek o dodatkowe modyfikacje algorytmiczne oraz nowe scenariusze testowe.

Z przedstawionego w poprzednich punktach materiału można wyciągnąć ogólny wniosek, że postawione w niniejszej pracy cele zostały spełnione.

7.2 Dalszy rozwój aplikacji

Uzyskane wyniki stwarzają podstawy do dalszego rozwoju opracowanego oprogramowania. Trzy potencjalne kierunki dalszego rozwoju aplikacji są następujące:

- opracowanie alternatywnych metod realizacji boostingu lokalnego,
- rozszerzenie scenariuszy testowych o kolejne zbiory danych, również o dane niegeograficzne,

- uogólnienie badanych zagadnień na problem regresji.

Pierwszy z powyższych kierunków zakłada wykorzystanie podejść odmiennych niż dodatkowy klasyfikator towarzyszący każdemu z klasyfikatorów bazowych. W grupie alternatywnych metod zawiera się między innymi wykorzystanie metod grupowania (analizy skupień) w celu dyskretnego partycjonowania przestrzeni przykładów.

Innym możliwym kierunkiem dalszych badań jest rozszerzenie eksperymentów na dane pozbawione znaczenia geograficznego, o zróżnicowanych charakterystykach. Celem takich eksperymentów byłoby dokładne wyznaczenie zakresu skuteczności wprowadzonych modyfikacji oraz zbadanie związku pomiędzy skutecznością klasyfikatorów grupowych a strukturą zbioru danych.

Ostatnia z proponowanych dróg rozwoju dla przedstawionych w niniejszej pracy koncepcji mogłaby być związana z zagadnieniem regresji. Ponieważ meta-algorytm boosting ma swoje zastosowanie również w tym problemie, wartościowym kierunkiem badań może być analiza skuteczności wariantów lokalnych w poprawie skuteczności modeli regresji. Uogólnienie to wymagałoby wykorzystania odmiennych metryk skuteczności modeli oraz opracowania metod implementacji boostingu lokalnego w nowym kontekście.

Literatura

- [1] A. Samuel, "Some studies in machine learning using the game of checkers," *IBM Journal of Research and Development*, Mar 1959.
- [2] S. Singh and N. Singh, "Big data analytics," in *Communication, Information Computing Technology (ICCICT), 2012 International Conference on*, pp. 1–4, Oct 2012.
- [3] D. Laney, "3D data management: Controlling data volume, velocity, and variety," tech. rep., META Group, February 2001.
- [4] Holler, Tsiatsis, Mulligan, Avesand, Karnouskos, Boyle, *From Machine-to-Machine to the Internet of Things: Introduction to a New Age of Intelligence*. Academic Press, 2014.
- [5] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *J. Comput. Syst. Sci.*, vol. 55, pp. 119–139, Aug. 1997.
- [6] R. E. Schapire, "The boosting approach to machine learning: An overview," 2002.
- [7] P. Cichosz, *Systemy uczące się*. Warszawa: Wydawnictwa Naukowo-Techniczne, 2000.
- [8] D. Kriesel, *A Brief Introduction to Neural Networks*. 2007.
- [9] S. O. Breiman, Friedman, *Classification and Regression Trees*. Wadsworth Statistics, 1984.
- [10] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, 1995.
- [11] D. Wolpert and W. Macready, "No free lunch theorems for optimization," *Evolutionary Computation, IEEE Transactions on*, vol. 1, pp. 67–82, Apr 1997.

- [12] Mulawka, J.J., *Systemy ekspertowe*. Wydawnictwa Naukowo Techniczne, 1997.
- [13] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [14] C. J. Van Rijsbergen, *Information Retrieval*. Butterworth, 1979.
- [15] L. Breiman, "Bagging predictors," in *Machine Learning*, pp. 123–140, 1996.
- [16] Y. Freund, "An adaptive version of the boost by majority algorithm," in *In Proceedings of the Twelfth Annual Conference on Computational Learning Theory*, pp. 102–113, 2000.
- [17] J. Friedman, T. Hastie, and R. Tibshirani, "Additive logistic regression: a statistical view of boosting," *Annals of Statistics*, vol. 28, p. 2000, 1998.
- [18] A. Demiriz, K. P. Bennett, and J. Shawe-Taylor, "Linear programming boosting via column generation," *Mach. Learn.*, vol. 46, no. 1-3, pp. 225–254, 2002.
- [19] L. Mason, J. Baxter, P. Bartlett, and M. Frean, "Boosting algorithms as gradient descent in function space," 1999.
- [20] S. B. Kotsiantis and P. E. Pintelas, "Local voting of weak classifiers," *KES Journal*, vol. 9, no. 3, pp. 239–248, 2005.
- [21] C. G. Atkeson, A. W. Moore, and S. Schaal, "Locally weighted learning for control.," *Artif. Intell. Rev.*, vol. 11, no. 1-5, pp. 75–113, 1997.
- [22] B. Pace, "Sparse spatial autoregressions.," *Statistics and Probability Letters*, 1997.